

QuantumTraderAI

Generated by Doxygen 1.14.0

Chapter 1

QuantumTraderAI: Autonomous AI Trading Empire

QuantumTraderAI is an elite, modular, scalable AI trading engine designed to start at \$100 and scale into millions. Powered by FastAPI, LSTM, Transformers, Redis, and Docker.

1.1 Deployments

1.1.1 Backend (Railway)

1. Push this repo to GitHub
 2. Connect GitHub to [Railway](#)
 3. Add your `.env` variables:
 - TELEGRAM_TOKEN
 - TELEGRAM_CHAT_ID
 - TWELVE_DATA_API_KEY
 4. Railway builds your FastAPI + Redis stack automatically
-

1.1.2 Frontend (Vercel)

1. Deploy `/frontend/` folder via [Vercel](#)
 2. Set the root to `frontend`
 3. Your live dashboard is now accessible globally
-

1.2 Features

- AI Strategies: Momentum, Reversion, LSTM, Transformer
 - Meta-Learner: Self-tunes strategies based on PnL history
 - Redis + Celery + Docker
 - Mobile Command Center (React Native/Expo)
 - Push Notifications + Telegram Alerts
-

1.3 Mobile App (Optional)

Use `QuantumTraderMobile` folder with Expo to deploy the native app:

- Biometric unlock
 - Signal charts
 - Live trade trigger
 - Push alert viewer
-

1.4 Folder Structure

```
QuantumTraderAI/  
  backend/  
    main.py  
    ai_models/meta_learner.py  
    strategies/  
    utils/  
  frontend/  
    index.html  
  docker-compose.yml  
  requirements.txt  
  .env.example
```

1.5 Meta-Learning Example

```
from backend.ai_models.meta_learner import meta  
  
print(meta.recommend_strategies())  
print(meta.confidence_weight("LSTM"))
```

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

allocator	??
backtester	??
backtesting	??
celery_config	??
celery_worker	??
cost_analysis	??
data_loader	??
feature_engineering	??
latency_optimizer	??
legal	??
lstm_model	??
main	??
manual_trade	??
market_predictor	??
market_simulation	??
mean_reversion	??
meta_learner	??
meta_reinforcement	??
metrics	??
momentum	??
nlp_sentiment	??
portfolio_optimizer	??
real_time_analysis	??
redis_client	??
reporting	??
risk_management	??
risk_manager	??
send_push	??
settings	??
strategy_framework	??
strategy_runner	??
telegram_alerts	??
test_real_time_analysis	??
test_reporting	??
trade_executor	??
train_lstm	??
transformer_model	??
twelve_data	??
websocket_stream	??

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

backtesting.Backtester	??
Base	
main.PerformanceMetric	??
main.RiskMetric	??
main.Trade	??
BaseSettings	
settings.Settings	??
main.CacheManager	??
feature_engineering.FeatureEngineer	??
Layer	
transformer_model.MultiHeadSelfAttention	??
transformer_model.TransformerBlock	??
legal.LegalProtector	??
lstm_model.LSTMModel	??
market_predictor.MarketPredictor	??
market_simulation.MarketSimulator	??
meta_learner.MetaLearner	??
meta_reinforcement.MetaReinforcementLearner	??
metrics.Metrics	??
main.PerformanceMonitor	??
portfolio_optimizer.PortfolioOptimizer	??
main.RateLimiter	??
real_time_analysis.RealTimeAnalyzer	??
main.RequestValidator	??
risk_management.RiskManager	??
risk_manager.RiskManager	??
strategy_framework.StrategyFramework	??
unittest.TestCase	
test_real_time_analysis.TestRealTimeAnalyzer	??
test_reporting.TestReporter	??
strategy_framework.TradingStrategy	??
cost_analysis.TransactionCostAnalyzer	??
transformer_model.TransformerModel	??

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

backtesting.Backtester	??
main.CacheManager	??
feature_engineering.FeatureEngineer	??
legal.LegalProtector	??
lstm_model.LSTMModel	??
market_predictor.MarketPredictor	??
market_simulation.MarketSimulator	??
meta_learner.MetaLearner	??
meta_reinforcement.MetaReinforcementLearner	??
metrics.Metrics	??
transformer_model.MultiHeadSelfAttention	??
main.PerformanceMetric	??
main.PerformanceMonitor	??
portfolio_optimizer.PortfolioOptimizer	??
main.RateLimiter	??
real_time_analysis.RealTimeAnalyzer	??
main.RequestValidator	??
risk_management.RiskManager	??
risk_manager.RiskManager	??
main.RiskMetric	??
settings.Settings	??
strategy_framework.StrategyFramework	??
test_real_time_analysis.TestRealTimeAnalyzer	??
test_reporting.TestReporter	??
main.Trade	??
strategy_framework.TradingStrategy	??
cost_analysis.TransactionCostAnalyzer	??
transformer_model.TransformerBlock	??
transformer_model.TransformerModel	??

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

backend/backtesting.py	??
backend/cost_analysis.py	??
backend/main.py	??
backend/market_simulation.py	??
backend/metrics.py	??
backend/portfolio_optimizer.py	??
backend/real_time_analysis.py	??
backend/reporting.py	??
backend/risk_management.py	??
backend/ai_models/allocator.py	??
backend/ai_models/feature_engineering.py	??
backend/ai_models/lstm_model.py	??
backend/ai_models/market_predictor.py	??
backend/ai_models/meta_learner.py	??
backend/ai_models/meta_reinforcement.py	??
backend/ai_models/portfolio_optimizer.py	??
backend/ai_models/risk_manager.py	??
backend/ai_models/train_lstm.py	??
backend/ai_models/transformer_model.py	??
backend/api/manual_trade.py	??
backend/config/celery_config.py	??
backend/config/settings.py	??
backend/data/websocket_stream.py	??
backend/integrations/nlp_sentiment.py	??
backend/integrations/telegram_alerts.py	??
backend/integrations/twelve_data.py	??
backend/strategies/mean_reversion.py	??
backend/strategies/momentum.py	??
backend/strategies/strategy_framework.py	??
backend/tasks/backtester.py	??
backend/tasks/celery_worker.py	??
backend/tasks/strategy_runner.py	??
backend/tests/test_real_time_analysis.py	??
backend/tests/test_reporting.py	??
backend/utils/data_loader.py	??
backend/utils/latency_optimizer.py	??
backend/utils/legal.py	??
backend/utils/redis_client.py	??
backend/utils/send_push.py	??
backend/utils/trade_executor.py	??

Chapter 6

Namespace Documentation

6.1 allocator Namespace Reference

6.2 backtester Namespace Reference

Functions

- [run_backtest](#) (data, strategy_fn, strategy_name, output_path='data/backtest_results.json')

6.2.1 Function Documentation

6.2.1.1 run_backtest()

```
backtester.run_backtest (
    data,
    strategy_fn,
    strategy_name,
    output_path = 'data/backtest_results.json')
```

6.3 backtesting Namespace Reference

Classes

- class [Backtester](#)

Variables

- [logger](#) = logging.getLogger(__name__)
- [backtester](#) = [Backtester](#)()

6.3.1 Variable Documentation

6.3.1.1 backtester

```
backtesting.backtester = Backtester()
```

6.3.1.2 logger

```
backtesting.logger = logging.getLogger(__name__)
```

6.4 celery_config Namespace Reference

6.5 celery_worker Namespace Reference

6.6 cost_analysis Namespace Reference

Classes

- class [TransactionCostAnalyzer](#)

Functions

- float [_fast_commission_cost](#) (float trade_size, float price, float base_commission)
- float [_fast_slippage](#) (float trade_size, float price, float market_volume, float slippage_base, float liquidity_threshold)
- float [_fast_market_impact](#) (float trade_size, float price, float market_volume, float market_impact_exponent)
- float [_fast_total_transaction_cost](#) (float trade_size, float price, float market_volume, float base_commission, float slippage_base, float liquidity_threshold, float market_impact_exponent)

Variables

- [logger](#) = logging.getLogger(__name__)
- dict [cost_cache](#) = {}
- [last_cache_update](#) = datetime.min
- int [CACHE_TTL](#) = 300
- [cost_analyzer](#) = [TransactionCostAnalyzer](#)()

6.6.1 Function Documentation

6.6.1.1 _fast_commission_cost()

```
float cost_analysis._fast_commission_cost (
    float trade_size,
    float price,
    float base_commission) [protected]
```

Fast commission cost calculation using Numba

6.6.1.2 _fast_market_impact()

```
float cost_analysis._fast_market_impact (
    float trade_size,
    float price,
    float market_volume,
    float market_impact_exponent) [protected]
```

Fast market impact calculation using Numba

6.6.1.3 _fast_slippage()

```
float cost_analysis._fast_slippage (
    float trade_size,
    float price,
    float market_volume,
    float slippage_base,
    float liquidity_threshold) [protected]
```

Fast slippage calculation using Numba

6.6.1.4 `_fast_total_transaction_cost()`

```
float cost_analysis._fast_total_transaction_cost (  
    float trade_size,  
    float price,  
    float market_volume,  
    float base_commission,  
    float slippage_base,  
    float liquidity_threshold,  
    float market_impact_exponent) [protected]
```

Fast total transaction cost calculation using Numba

6.6.2 Variable Documentation

6.6.2.1 `CACHE_TTL`

```
int cost_analysis.CACHE_TTL = 300
```

6.6.2.2 `cost_analyzer`

```
cost_analysis.cost_analyzer = TransactionCostAnalyzer\(\)
```

6.6.2.3 `cost_cache`

```
dict cost_analysis.cost_cache = {}
```

6.6.2.4 `last_cache_update`

```
cost_analysis.last_cache_update = datetime.min
```

6.6.2.5 `logger`

```
cost_analysis.logger = logging.getLogger(__name__)
```

6.7 data_loader Namespace Reference

Functions

- [load_realtime_data\(\)](#)

6.7.1 Function Documentation

6.7.1.1 `load_realtime_data()`

```
data_loader.load_realtime_data ()
```

6.8 feature_engineering Namespace Reference

Classes

- class [FeatureEngineer](#)

Variables

- `logger` = `logging.getLogger(__name__)`
- `feature_engineer` = [FeatureEngineer\(\)](#)

6.8.1 Variable Documentation

6.8.1.1 feature_engineer

```
feature_engineering.feature_engineer = FeatureEngineer()
```

6.8.1.2 logger

```
feature_engineering.logger = logging.getLogger(__name__)
```

6.9 latency_optimizer Namespace Reference

Functions

- [measure_latency\(\)](#)

Variables

- dict [API_ENDPOINTS](#)

6.9.1 Function Documentation

6.9.1.1 measure_latency()

```
latency_optimizer.measure_latency()
```

6.9.2 Variable Documentation

6.9.2.1 API_ENDPOINTS

```
dict latency_optimizer.API_ENDPOINTS
```

Initial value:

```
00001 = {
00002     "twelvedata": "https://api.twelvedata.com/time_series?symbol=AAPL&interval=1min",
00003     "yahoo": "https://query1.finance.yahoo.com/v7/finance/quote?symbols=AAPL"
00004 }
```

6.10 legal Namespace Reference

Classes

- class [LegalProtector](#)

6.11 lstm_model Namespace Reference

Classes

- class [LSTMModel](#)

Variables

- [logger](#) = logging.getLogger(__name__)
- [lstm](#)

6.11.1 Variable Documentation

6.11.1.1 logger

```
lstm_model.logger = logging.getLogger(__name__)
```


6.11.1.2 lstm

`lstm_model.lstm`

Initial value:

```
00001 = LSTMModel(
00002     input_shape=(60, 10), # 60 timesteps, 10 features
00003     dropout_rate=0.2,
00004     learning_rate=0.001,
00005     batch_size=32,
00006     epochs=100,
00007     patience=10
00008 )
```

6.12 main Namespace Reference

Classes

- class [CacheManager](#)
- class [PerformanceMetric](#)
- class [PerformanceMonitor](#)
- class [RateLimiter](#)
- class [RequestValidator](#)
- class [RiskMetric](#)
- class [Trade](#)

Functions

- [log_ai_decision](#) (str decision_type, dict details)
- [bool check_ai_safety](#) (dict trade_details)
- [bool require_user_confirmation](#) (dict decision)
- [ping_connection](#) (dbapi_connection, connection_record, connection_proxy)
- [get_db](#) ()
- [get_cache](#) (namespace='default')
- [get_redis_pool](#) ()
- [get_memcached_client](#) ()
- [custom_exception_handler](#) (Request request, Exception exc)
- [add_security_headers](#) (Request request, call_next)
- [monitor_performance](#) (Request request, call_next)
- [update_metrics](#) (str endpoint, float response_time)
- [circuit_breaker](#) (fn, *args, **kwargs)
- [run_background_task](#) (task)
- [validate_request](#) (Request request, call_next)
- [add_process_time_header](#) (Request request, call_next)
- [health_check](#) ()
- [get_metrics](#) ()
- [read_root](#) ()
- [run_all_strategies](#) ()

Variables

- dict [AI_CONFIG](#)
- dict [ai_safety_monitor](#)
- [level](#)
- [format](#)
- [handlers](#)
- [datefmt](#)
- [logger](#) = logging.getLogger("quantum_trader")
- [jaeger_exporter](#)
- [span_processor](#) = BatchSpanProcessor([jaeger_exporter](#))

- str `DATABASE_URL` = "postgresql+asyncpg://user:password@localhost/quantum_trader"
- engine
- SessionLocal
- Base = declarative_base()
- dict `CACHE_CONFIG`
- app
- int `RATE_LIMIT_WINDOW` = 60
- int `RATE_LIMIT_REQUESTS` = 100
- rate_limit_cache = TTLCache(maxsize=1000, ttl=`RATE_LIMIT_WINDOW`)
- SECRET_KEY = secrets.token_hex(32)
- str `ALGORITHM` = "HS256"
- int `ACCESS_TOKEN_EXPIRE_MINUTES` = 30
- pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
- redis_pool = None
- memcached_client = None
- dsn
- traces_sample_rate
- environment
- integrations
- before_send
- int `CIRCUIT_TIMEOUT` = 30000
- int `last_failure_time` = 0
- bool `circuit_open` = False
- background_tasks = set()
- cache_manager = CacheManager()
- allow_origins
- allow_credentials
- allow_methods
- allow_headers
- expose_headers
- allow_origin_regex
- max_age
- minimum_size
- compresslevel
- server_side
- client_side
- allowed_hosts
- except_hosts
- status_code
- ssl_host
- ssl_port
- secret_key
- session_cookie
- same_site
- https_only
- cookie_domain
- headers
- max_requests
- window
- key_func
- storage
- on_limit_exceeded
- threshold
- metrics_store
- on_slow_request

- [backend](#)
- [on_error](#)
- [cache](#)
- [cache_key_func](#)
- [cache_ttl](#)
- [cache_bypass_func](#)
- [tracer](#)
- [span_name_func](#)
- [trace_id_header](#)
- [span_id_header](#)
- [log_format](#)
- [log_level](#)
- [include_headers](#)
- [include_body](#)
- [validators](#)
- [failure_threshold](#)
- [reset_timeout](#)
- [on_circuit_open](#)
- dict [market_data_cache](#) = {}
- [last_cache_update](#) = datetime.min
- int [CACHE_TTL](#) = 300

6.12.1 Function Documentation

6.12.1.1 `add_process_time_header()`

```
main.add_process_time_header (  
    Request request,  
    call_next)
```

6.12.1.2 `add_security_headers()`

```
main.add_security_headers (  
    Request request,  
    call_next)
```

6.12.1.3 `check_ai_safety()`

```
bool main.check_ai_safety (  
    dict trade_details)
```

Check if AI decision is within safety parameters

6.12.1.4 `circuit_breaker()`

```
main.circuit_breaker (  
    fn,  
    * args,  
    ** kwargs)
```

6.12.1.5 `custom_exception_handler()`

```
main.custom_exception_handler (  
    Request request,  
    Exception exc)
```

6.12.1.6 get_cache()

```
main.get_cache (
    namespace = 'default')
```

6.12.1.7 get_db()

```
main.get_db ()
```

6.12.1.8 get_memcached_client()

```
main.get_memcached_client ()
```

6.12.1.9 get_metrics()

```
main.get_metrics ()
```

6.12.1.10 get_redis_pool()

```
main.get_redis_pool ()
```

6.12.1.11 health_check()

```
main.health_check ()
```

6.12.1.12 log_ai_decision()

```
main.log_ai_decision (
    str decision_type,
    dict details)

Log AI decisions with full context
```

6.12.1.13 monitor_performance()

```
main.monitor_performance (
    Request request,
    call_next)
```

6.12.1.14 ping_connection()

```
main.ping_connection (
    dbapi_connection,
    connection_record,
    connection_proxy)
```

6.12.1.15 read_root()

```
main.read_root ()
```

6.12.1.16 require_user_confirmation()

```
bool main.require_user_confirmation (
    dict decision)

Require user confirmation for AI decisions
```

6.12.1.17 run_all_strategies()

```
main.run_all_strategies ()
```

6.12.1.18 run_background_task()

```
main.run_background_task (
    task)
```

6.12.1.19 update_metrics()

```
main.update_metrics (
    str endpoint,
    float response_time)
```

6.12.1.20 validate_request()

```
main.validate_request (
    Request request,
    call_next)
```

6.12.2 Variable Documentation**6.12.2.1 ACCESS_TOKEN_EXPIRE_MINUTES**

```
int main.ACCESS_TOKEN_EXPIRE_MINUTES = 30
```

6.12.2.2 AI_CONFIG

```
dict main.AI_CONFIG
```

Initial value:

```
00001 = {
00002     'max_strategy_weight': 0.3, # Maximum weight any single strategy can have
00003     'min_strategy_weight': 0.05, # Minimum weight any strategy can have
00004     'max_position_size': 0.05, # Maximum position size as % of portfolio
00005     'require_user_confirmation': False, # AI makes decisions autonomously
00006     'log_level': 'INFO', # Detailed logging of AI activities
00007     'max_consecutive_losses': 5, # Maximum consecutive losses before AI pause
00008     'max_drawdown': 0.10, # Maximum allowed drawdown before AI pause
00009     'risk_tolerance': 0.02, # Maximum daily risk per trade
00010     'profit_target': 1.5, # Target profit multiple
00011     'stop_loss': 0.5, # Stop loss multiple
00012     'max_leverage': 10, # Maximum leverage allowed
00013     'max_trades': 50, # Maximum number of concurrent trades
00014     'portfolio_rebalance_freq': '1H', # Portfolio rebalancing frequency
00015     'market_data_window': '24H', # Historical data window for analysis
00016     'risk_management': {
00017         'volatility_adjustment': True,
00018         'position_sizing': 'KellyCriterion',
00019         'diversification_factor': 0.8
00020     }
00021 }
```

6.12.2.3 ai_safety_monitor

```
dict main.ai_safety_monitor
```

Initial value:

```
00001 = {
00002     'consecutive_losses': 0,
00003     'current_drawdown': 0,
00004     'last_user_confirmation': None,
00005     'active': True,
00006     'last_decision': None
00007 }
```

6.12.2.4 ALGORITHM

```
str main.ALGORITHM = "HS256"
```

6.12.2.5 allow_credentials

```
main.allow_credentials
```

6.12.2.6 allow_headers

main.allow_headers

6.12.2.7 allow_methods

main.allow_methods

6.12.2.8 allow_origin_regex

main.allow_origin_regex

6.12.2.9 allow_origins

main.allow_origins

6.12.2.10 allowed_hosts

main.allowed_hosts

6.12.2.11 app

main.app

Initial value:

```
00001 = FastAPI(  
00002     title="Quantum Trader AI API",  
00003     description="Advanced trading platform with AI-powered strategies",  
00004     version="1.0.0",  
00005     docs_url="/docs",  
00006     redoc_url="/redoc",  
00007     openapi_url="/api/openapi.json",  
00008     default_response_class=JSONResponse  
00009 )
```

6.12.2.12 backend

main.backend

6.12.2.13 background_tasks

main.background_tasks = set()

6.12.2.14 Base

main.Base = declarative_base()

6.12.2.15 before_send

main.before_send

6.12.2.16 cache

main.cache

6.12.2.17 cache_bypass_func

main.cache_bypass_func

6.12.2.18 CACHE_CONFIG

```
dict main.CACHE_CONFIG
```

Initial value:

```
00001 = {
00002     'default': {
00003         'cache': 'aiocache.RedisCache',
00004         'endpoint': 'localhost',
00005         'port': 6379,
00006         'ttl': 300,
00007         'timeout': 1
00008     },
00009     'market_data': {
00010         'cache': 'aiocache.RedisCache',
00011         'endpoint': 'localhost',
00012         'port': 6379,
00013         'ttl': 60,
00014         'timeout': 0.5
00015     },
00016     'performance': {
00017         'cache': 'aiocache.RedisCache',
00018         'endpoint': 'localhost',
00019         'port': 6379,
00020         'ttl': 3600,
00021         'timeout': 1
00022     }
00023 }
```

6.12.2.19 cache_key_func

```
main.cache_key_func
```

6.12.2.20 cache_manager

```
main.cache_manager = CacheManager()
```

6.12.2.21 CACHE_TTL

```
int main.CACHE_TTL = 300
```

6.12.2.22 cache_ttl

```
main.cache_ttl
```

6.12.2.23 circuit_open

```
bool main.circuit_open = False
```

6.12.2.24 CIRCUIT_TIMEOUT

```
int main.CIRCUIT_TIMEOUT = 30000
```

6.12.2.25 client_side

```
main.client_side
```

6.12.2.26 compresslevel

```
main.compresslevel
```

6.12.2.27 cookie_domain

```
main.cookie_domain
```

6.12.2.28 DATABASE_URL

```
str main.DATABASE_URL = "postgresql+asyncpg://user:password@localhost/quantum_trader"
```

6.12.2.29 datefmt

main.datefmt

6.12.2.30 dsn

main.dsn

6.12.2.31 engine

main.engine

Initial value:

```
00001 = create_async_engine(  
00002     DATABASE_URL,  
00003     pool_pre_ping=True,  
00004     pool_size=10,  
00005     max_overflow=20,  
00006     pool_timeout=30,  
00007     pool_recycle=3600,  
00008     echo=False  
00009 )
```

6.12.2.32 environment

main.environment

6.12.2.33 except_hosts

main.except_hosts

6.12.2.34 expose_headers

main.expose_headers

6.12.2.35 failure_threshold

main.failure_threshold

6.12.2.36 format

main.format

6.12.2.37 handlers

main.handlers

6.12.2.38 headers

main.headers

6.12.2.39 https_only

main.https_only

6.12.2.40 include_body

main.include_body

6.12.2.41 include_headers

main.include_headers

6.12.2.42 integrations

`main.integrations`

6.12.2.43 jaeger_exporter

`main.jaeger_exporter`

Initial value:

```
00001 = JaegerExporter(  
00002     agent_host_name='localhost',  
00003     agent_port=6831,  
00004 )
```

6.12.2.44 key_func

`main.key_func`

6.12.2.45 last_cache_update

`main.last_cache_update = datetime.min`

6.12.2.46 last_failure_time

`int main.last_failure_time = 0`

6.12.2.47 level

`main.level`

6.12.2.48 log_format

`main.log_format`

6.12.2.49 log_level

`main.log_level`

6.12.2.50 logger

`main.logger = logging.getLogger("quantum_trader")`

6.12.2.51 market_data_cache

`dict main.market_data_cache = {}`

6.12.2.52 max_age

`main.max_age`

6.12.2.53 max_requests

`main.max_requests`

6.12.2.54 memcached_client

`main.memcached_client = None`

6.12.2.55 metrics_store

`main.metrics_store`

6.12.2.56 minimum_size

```
main.minimum_size
```

6.12.2.57 on_circuit_open

```
main.on_circuit_open
```

6.12.2.58 on_error

```
main.on_error
```

6.12.2.59 on_limit_exceeded

```
main.on_limit_exceeded
```

6.12.2.60 on_slow_request

```
main.on_slow_request
```

6.12.2.61 pwd_context

```
main.pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
```

6.12.2.62 rate_limit_cache

```
main.rate_limit_cache = TTLCache(maxsize=1000, ttl=RATE_LIMIT_WINDOW)
```

6.12.2.63 RATE_LIMIT_REQUESTS

```
int main.RATE_LIMIT_REQUESTS = 100
```

6.12.2.64 RATE_LIMIT_WINDOW

```
int main.RATE_LIMIT_WINDOW = 60
```

6.12.2.65 redis_pool

```
main.redis_pool = None
```

6.12.2.66 reset_timeout

```
main.reset_timeout
```

6.12.2.67 same_site

```
main.same_site
```

6.12.2.68 SECRET_KEY

```
main.SECRET_KEY = secrets.token_hex(32)
```

6.12.2.69 secret_key

```
main.secret_key
```

6.12.2.70 server_side

```
main.server_side
```

6.12.2.71 session_cookie

main.session_cookie

6.12.2.72 SessionLocal

main.SessionLocal

Initial value:

```
00001 = sessionmaker(  
00002     autocommit=False,  
00003     autoflush=False,  
00004     bind=engine,  
00005     class_=AsyncSession  
00006 )
```

6.12.2.73 span_id_header

main.span_id_header

6.12.2.74 span_name_func

main.span_name_func

6.12.2.75 span_processor

main.span_processor = BatchSpanProcessor([jaeger_exporter](#))

6.12.2.76 ssl_host

main.ssl_host

6.12.2.77 ssl_port

main.ssl_port

6.12.2.78 status_code

main.status_code

6.12.2.79 storage

main.storage

6.12.2.80 threshold

main.threshold

6.12.2.81 trace_id_header

main.trace_id_header

6.12.2.82 tracer

main.tracer

6.12.2.83 traces_sample_rate

main.traces_sample_rate

6.12.2.84 validators

main.validators

6.12.2.85 window

`main.window`

6.13 manual_trade Namespace Reference

Functions

- [manual_trade](#) (Request request)

Variables

- [router](#) = `APIRouter()`

6.13.1 Function Documentation

6.13.1.1 manual_trade()

```
manual_trade.manual_trade (
    Request request)
```

6.13.2 Variable Documentation

6.13.2.1 router

```
manual_trade.router = APIRouter()
```

6.14 market_predictor Namespace Reference

Classes

- class [MarketPredictor](#)

Variables

- [logger](#) = `logging.getLogger(__name__)`
- [market_predictor](#) = `MarketPredictor()`

6.14.1 Variable Documentation

6.14.1.1 logger

```
market_predictor.logger = logging.getLogger(__name__)
```

6.14.1.2 market_predictor

```
market_predictor.market_predictor = MarketPredictor()
```

6.15 market_simulation Namespace Reference

Classes

- class [MarketSimulator](#)

Variables

- [logger](#) = `logging.getLogger(__name__)`
- [market_simulator](#) = `MarketSimulator()`

6.15.1 Variable Documentation

6.15.1.1 logger

```
market_simulation.logger = logging.getLogger(__name__)
```

6.15.1.2 market_simulator

```
market_simulation.market_simulator = MarketSimulator()
```

6.16 mean_reversion Namespace Reference

Functions

- [mean_reversion_strategy](#) (data)

6.16.1 Function Documentation

6.16.1.1 mean_reversion_strategy()

```
mean_reversion.mean_reversion_strategy (
    data)
```

6.17 meta_learner Namespace Reference

Classes

- class [MetaLearner](#)

Variables

- [logger](#) = logging.getLogger(__name__)
- [meta](#)

6.17.1 Variable Documentation

6.17.1.1 logger

```
meta_learner.logger = logging.getLogger(__name__)
```

6.17.1.2 meta

```
meta_learner.meta
```

Initial value:

```
00001 = MetaLearner(
00002     history_file='data/trade_history.json',
00003     lookback_window=60,
00004     confidence_threshold=0.7,
00005     min_samples=20
00006 )
```

6.18 meta_reinforcement Namespace Reference

Classes

- class [MetaReinforcementLearner](#)

6.19 metrics Namespace Reference

Classes

- class [Metrics](#)

Functions

- float [_fast_sharpe_ratio](#) (np.ndarray returns, float risk_free_rate)
- float [_fast_max_drawdown](#) (np.ndarray cumulative)
- Dict [_fast_basic_metrics](#) (np.ndarray returns, float risk_free_rate)

Variables

- [logger](#) = logging.getLogger(__name__)
- dict [metrics_cache](#) = {}
- [last_cache_update](#) = datetime.min
- int [CACHE_TTL](#) = 300

6.19.1 Function Documentation

6.19.1.1 _fast_basic_metrics()

```
Dict metrics._fast_basic_metrics (
    np.ndarray returns,
    float risk_free_rate) [protected]
```

Fast basic metrics calculation using Numba

6.19.1.2 _fast_max_drawdown()

```
float metrics._fast_max_drawdown (
    np.ndarray cumulative) [protected]
```

Fast maximum drawdown calculation using Numba

6.19.1.3 _fast_sharpe_ratio()

```
float metrics._fast_sharpe_ratio (
    np.ndarray returns,
    float risk_free_rate) [protected]
```

Fast Sharpe ratio calculation using Numba

6.19.2 Variable Documentation

6.19.2.1 CACHE_TTL

```
int metrics.CACHE_TTL = 300
```

6.19.2.2 last_cache_update

```
metrics.last_cache_update = datetime.min
```

6.19.2.3 logger

```
metrics.logger = logging.getLogger(__name__)
```

6.19.2.4 metrics_cache

```
dict metrics.metrics_cache = {}
```

6.20 momentum Namespace Reference

Functions

- [momentum_strategy](#) (data)

6.20.1 Function Documentation

6.20.1.1 momentum_strategy()

```
momentum.momentum_strategy (  
    data)
```

6.21 nlp_sentiment Namespace Reference

Functions

- [analyze_foreign_news](#) (texts, target_lang='en')

Variables

- [translator](#) = Translator()
- [sentiment_pipeline](#) = pipeline("sentiment-analysis")

6.21.1 Function Documentation

6.21.1.1 analyze_foreign_news()

```
nlp_sentiment.analyze_foreign_news (  
    texts,  
    target_lang = 'en')
```

6.21.2 Variable Documentation

6.21.2.1 sentiment_pipeline

```
nlp_sentiment.sentiment_pipeline = pipeline("sentiment-analysis")
```

6.21.2.2 translator

```
nlp_sentiment.translator = Translator()
```

6.22 portfolio_optimizer Namespace Reference

Classes

- class [PortfolioOptimizer](#)

Variables

- [logger](#) = logging.getLogger(__name__)
- [portfolio_optimizer](#) = [PortfolioOptimizer](#)()

6.22.1 Variable Documentation

6.22.1.1 logger

```
portfolio_optimizer.logger = logging.getLogger(__name__)
```

6.22.1.2 portfolio_optimizer

```
portfolio_optimizer.portfolio_optimizer = PortfolioOptimizer()
```

6.23 real_time_analysis Namespace Reference

Classes

- class [RealTimeAnalyzer](#)

Variables

- [logger](#) = logging.getLogger(__name__)
- [real_time_analyzer](#) = [RealTimeAnalyzer](#)()

6.23.1 Variable Documentation

6.23.1.1 logger

```
real_time_analysis.logger = logging.getLogger(__name__)
```

6.23.1.2 real_time_analyzer

```
real_time_analysis.real_time_analyzer = RealTimeAnalyzer()
```

6.24 redis_client Namespace Reference

6.25 reporting Namespace Reference

Variables

- [logger](#) = logging.getLogger(__name__)

6.25.1 Variable Documentation

6.25.1.1 logger

```
reporting.logger = logging.getLogger(__name__)
```

6.26 risk_management Namespace Reference

Classes

- class [RiskManager](#)

Variables

- [logger](#) = logging.getLogger(__name__)
- [risk_manager](#) = [RiskManager](#)()

6.26.1 Variable Documentation

6.26.1.1 logger

```
risk_management.logger = logging.getLogger(__name__)
```

6.26.1.2 risk_manager

```
risk_management.risk_manager = RiskManager()
```

6.27 risk_manager Namespace Reference

Classes

- class [RiskManager](#)

Variables

- [logger](#) = logging.getLogger(__name__)
- [risk_manager](#) = [RiskManager](#)()

6.27.1 Variable Documentation

6.27.1.1 logger

```
risk_manager.logger = logging.getLogger(__name__)
```

6.27.1.2 risk_manager

```
risk_manager.risk_manager = RiskManager()
```

6.28 send_push Namespace Reference

Functions

- [send_mobile_push](#) (title, body)

6.28.1 Function Documentation

6.28.1.1 send_mobile_push()

```
send_push.send_mobile_push (  
    title,  
    body)
```

6.29 settings Namespace Reference

Classes

- class [Settings](#)

Variables

- [settings](#) = [Settings](#)()

6.29.1 Variable Documentation

6.29.1.1 settings

```
settings.settings = Settings()
```

6.30 strategy_framework Namespace Reference

Classes

- class [StrategyFramework](#)
- class [TradingStrategy](#)

Variables

- [logger](#) = logging.getLogger(__name__)

6.30.1 Variable Documentation

6.30.1.1 logger

```
strategy_framework.logger = logging.getLogger(__name__)
```

6.31 strategy_runner Namespace Reference

6.32 telegram_alerts Namespace Reference

Functions

- [send_telegram_message](#) (str message)

6.32.1 Function Documentation

6.32.1.1 send_telegram_message()

```
telegram_alerts.send_telegram_message (  
    str message)
```

6.33 test_real_time_analysis Namespace Reference

Classes

- class [TestRealTimeAnalyzer](#)

6.34 test_reporting Namespace Reference

Classes

- class [TestReporter](#)

6.35 trade_executor Namespace Reference

Functions

- [execute_trade](#) (signals)

6.35.1 Function Documentation

6.35.1.1 execute_trade()

```
trade_executor.execute_trade (  
    signals)
```

6.36 train_lstm Namespace Reference

Functions

- [train_lstm_model](#) (prices)

6.36.1 Function Documentation

6.36.1.1 train_lstm_model()

```
train_lstm.train_lstm_model (  
    prices)
```

6.37 transformer_model Namespace Reference

Classes

- class [MultiHeadSelfAttention](#)
- class [TransformerBlock](#)
- class [TransformerModel](#)

Variables

- [logger](#) = logging.getLogger(__name__)
- [transformer](#)

6.37.1 Variable Documentation

6.37.1.1 logger

```
transformer_model.logger = logging.getLogger(__name__)
```

6.37.1.2 transformer

```
transformer_model.transformer
```

Initial value:

```
00001 = TransformerModel(  
00002     input_shape=(60, 10), # 60 timesteps, 10 features  
00003     embed_dim=64,  
00004     num_heads=4,  
00005     ff_dim=32,  
00006     num_transformer_blocks=3,  
00007     dropout_rate=0.2,  
00008     learning_rate=0.001,  
00009     batch_size=32,  
00010     epochs=100,  
00011     patience=10  
00012 )
```

6.38 twelve_data Namespace Reference

Functions

- [fetch_realtime_data](#) (symbols)

6.38.1 Function Documentation

6.38.1.1 `fetch_realtime_data()`

```
twelve_data.fetch_realtime_data (  
    symbols)
```

6.39 websocket_stream Namespace Reference

Chapter 7

Class Documentation

7.1 backtesting.Backtester Class Reference

Public Member Functions

- `__init__` (self, float `initial_capital`=100000.0, float `commission`=0.001, float `slippage`=0.0005, int `max_positions`=10, float `risk_per_trade`=0.01, str `data_frequency`='daily')
- None `initialize_portfolio` (self, datetime start_date)
- float `calculate_position_size` (self, float current_price, float volatility)
- Dict `execute_trade` (self, str symbol, float price, float position_size, str direction)
- None `close_trade` (self, Dict trade, float exit_price)
- Dict[str, float] `calculate_metrics` (self)
- Dict `run_backtest` (self, callable strategy, pd.DataFrame data, datetime start_date, datetime end_date)
- None `plot_results` (self)

Public Attributes

- `initial_capital` = initial_capital
- `commission` = commission
- `slippage` = slippage
- `max_positions` = max_positions
- `risk_per_trade` = risk_per_trade
- `data_frequency` = data_frequency
- dict `portfolio` = None
- list `trades` = []
- dict `metrics` = {}
- int `current_position` = 0

7.1.1 Constructor & Destructor Documentation

7.1.1.1 `__init__()`

```
backtesting.Backtester.__init__ (
    self,
    float  initial_capital = 100000.0,
    float  commission     = 0.001,
    float  slippage        = 0.0005,
    int    max_positions   = 10,
    float  risk_per_trade  = 0.01,
    str    data_frequency  = 'daily')
```

Initialize backtesting framework

Args:

```
initial_capital: Starting capital
commission: Trading commission
slippage: Slippage percentage
max_positions: Maximum concurrent positions
risk_per_trade: Risk per trade
data_frequency: Data frequency
```

7.1.2 Member Function Documentation

7.1.2.1 calculate_metrics()

```
Dict[str, float] backtesting.Backtester.calculate_metrics (
    self)
```

Calculate backtesting metrics

7.1.2.2 calculate_position_size()

```
float backtesting.Backtester.calculate_position_size (
    self,
    float current_price,
    float volatility)
```

Calculate position size based on risk

7.1.2.3 close_trade()

```
None backtesting.Backtester.close_trade (
    self,
    Dict trade,
    float exit_price)
```

Close a trade position

7.1.2.4 execute_trade()

```
Dict backtesting.Backtester.execute_trade (
    self,
    str symbol,
    float price,
    float position_size,
    str direction)
```

Execute a trade

7.1.2.5 initialize_portfolio()

```
None backtesting.Backtester.initialize_portfolio (
    self,
    datetime start_date)
```

Initialize portfolio

7.1.2.6 plot_results()

```
None backtesting.Backtester.plot_results (
    self)
```

Plot backtest results

7.1.2.7 run_backtest()

```
Dict backtesting.Backtester.run_backtest (  
    self,  
    callable strategy,  
    pd.DataFrame data,  
    datetime start_date,  
    datetime end_date)
```

Run backtesting

7.1.3 Member Data Documentation

7.1.3.1 commission

```
backtesting.Backtester.commission = commission
```

7.1.3.2 current_position

```
int backtesting.Backtester.current_position = 0
```

7.1.3.3 data_frequency

```
backtesting.Backtester.data_frequency = data_frequency
```

7.1.3.4 initial_capital

```
backtesting.Backtester.initial_capital = initial_capital
```

7.1.3.5 max_positions

```
backtesting.Backtester.max_positions = max_positions
```

7.1.3.6 metrics

```
dict backtesting.Backtester.metrics = {}
```

7.1.3.7 portfolio

```
dict backtesting.Backtester.portfolio = None
```

7.1.3.8 risk_per_trade

```
backtesting.Backtester.risk_per_trade = risk_per_trade
```

7.1.3.9 slippage

```
backtesting.Backtester.slippage = slippage
```

7.1.3.10 trades

```
list backtesting.Backtester.trades = []
```

The documentation for this class was generated from the following file:

- backend/[backtesting.py](#)

7.2 main.CacheManager Class Reference

Public Member Functions

- [__init__](#) (self)
- [get](#) (self, str key)
- [set](#) (self, str key, Any value)
- [invalidate](#) (self, str key)

Public Attributes

- [cache](#) = TTLCache(maxsize=1000, ttl=300)
- [lock](#) = asyncio.Lock()

7.2.1 Constructor & Destructor Documentation

7.2.1.1 [__init__\(\)](#)

```
main.CacheManager.__init__ (  
    self)
```

7.2.2 Member Function Documentation

7.2.2.1 [get\(\)](#)

```
main.CacheManager.get (  
    self,  
    str key)
```

7.2.2.2 [invalidate\(\)](#)

```
main.CacheManager.invalidate (  
    self,  
    str key)
```

7.2.2.3 [set\(\)](#)

```
main.CacheManager.set (  
    self,  
    str key,  
    Any value)
```

7.2.3 Member Data Documentation

7.2.3.1 [cache](#)

```
main.CacheManager.cache = TTLCache(maxsize=1000, ttl=300)
```

7.2.3.2 [lock](#)

```
main.CacheManager.lock = asyncio.Lock()
```

The documentation for this class was generated from the following file:

- backend/[main.py](#)

7.3 feature_engineering.FeatureEngineer Class Reference

Public Member Functions

- `__init__` (self, List[int] `window_sizes`=[10, 20, 50, 100], List[str] `technical_indicators`=['RSI', 'MACD', 'BB', 'ADX', 'OBV'], List[str] `volatility_features`=['ATR', 'BBANDS'], List[str] `volume_features`=['OBV', 'MFI'], List[str] `momentum_features`=['RSI', 'ROC', 'ADX'], Dict `user_config`=None)
- `pd.DataFrame calculate_technical_indicators` (self, `pd.DataFrame df`)
- `pd.DataFrame calculate_volatility_features` (self, `pd.DataFrame df`)
- `pd.DataFrame calculate_volume_features` (self, `pd.DataFrame df`)
- `pd.DataFrame calculate_momentum_features` (self, `pd.DataFrame df`)
- `pd.DataFrame calculate_time_features` (self, `pd.DataFrame df`)
- `pd.DataFrame calculate_market_regime_features` (self, `pd.DataFrame df`)
- `pd.DataFrame normalize_features` (self, `pd.DataFrame df`)
- `pd.DataFrame create_feature_matrix` (self, `pd.DataFrame df`)
- List[str] `get_feature_columns` (self)

Public Attributes

- `window_sizes` = `window_sizes`
- `technical_indicators` = `technical_indicators`
- `volatility_features` = `volatility_features`
- `volume_features` = `volume_features`
- `momentum_features` = `momentum_features`
- `scaler` = `StandardScaler()`
- dict `feature_cache` = {}
- `last_update` = `None`
- int `max_features` = 100
- float `feature_importance_threshold` = 0.01
- int `memory_limit` = 1e9
- int `computation_time_limit` = 5
- `user_config` = `user_config` or {}
- bool `user_override` = `False`
- `feature_blacklist` = `set(self.user_config.get('blacklisted_features', []))`
- `feature_whitelist` = `set(self.user_config.get('whitelisted_features', []))`
- dict `feature_importance` = {}
- `last_feature_update` = `datetime.now()`
- dict `feature_calculation_times` = {}
- dict `memory_usage` = {}
- int `error_count` = 0
- int `max_errors` = 5
- `logger` = `logging.getLogger(__name__)`
- `feature_selector` = `None`
- `last_feature_selection` = `None`
- `feature_columns` = `df.columns.tolist()`

Protected Member Functions

- `_validate_configuration` (self)
- bool `_check_memory_usage` (self, `pd.DataFrame df`)
- bool `_check_computation_time` (self, float `start_time`)
- `_log_feature_calculation` (self, str `feature_name`, float `calculation_time`)
- `_handle_error` (self, Exception `error`, str `feature_name`)

7.3.1 Constructor & Destructor Documentation

7.3.1.1 `__init__()`

```
feature_engineering.FeatureEngineer.__init__ (
    self,
    List[int] window_sizes = [10, 20, 50, 100],
    List[str] technical_indicators = ['RSI', 'MACD', 'BB', 'ADX', 'OBV'],
    List[str] volatility_features = ['ATR', 'BBANDS'],
    List[str] volume_features = ['OBV', 'MFI'],
    List[str] momentum_features = ['RSI', 'ROC', 'ADX'],
    Dict user_config = None)
```

Initialize feature engineering pipeline with safety measures

Args:

```
window_sizes: List of window sizes for rolling calculations
technical_indicators: List of technical indicators to calculate
volatility_features: List of volatility features
volume_features: List of volume features
momentum_features: List of momentum features
user_config: User-defined configuration overrides
```

7.3.2 Member Function Documentation

7.3.2.1 `_check_computation_time()`

```
bool feature_engineering.FeatureEngineer._check_computation_time (
    self,
    float start_time) [protected]
```

Check if computation time is within limits

7.3.2.2 `_check_memory_usage()`

```
bool feature_engineering.FeatureEngineer._check_memory_usage (
    self,
    pd.DataFrame df) [protected]
```

Check if memory usage is within limits

7.3.2.3 `_handle_error()`

```
feature_engineering.FeatureEngineer._handle_error (
    self,
    Exception error,
    str feature_name) [protected]
```

Handle calculation errors with safety measures

7.3.2.4 `_log_feature_calculation()`

```
feature_engineering.FeatureEngineer._log_feature_calculation (
    self,
    str feature_name,
    float calculation_time) [protected]
```

Log feature calculation statistics

7.3.2.5 _validate_configuration()

```
feature_engineering.FeatureEngineer._validate_configuration (
    self) [protected]
```

Validate user configuration

7.3.2.6 calculate_market_regime_features()

```
pd.DataFrame feature_engineering.FeatureEngineer.calculate_market_regime_features (
    self,
    pd.DataFrame df)
```

Calculate market regime features

7.3.2.7 calculate_momentum_features()

```
pd.DataFrame feature_engineering.FeatureEngineer.calculate_momentum_features (
    self,
    pd.DataFrame df)
```

Calculate momentum indicators

7.3.2.8 calculate_technical_indicators()

```
pd.DataFrame feature_engineering.FeatureEngineer.calculate_technical_indicators (
    self,
    pd.DataFrame df)
```

Calculate technical indicators with safety checks

7.3.2.9 calculate_time_features()

```
pd.DataFrame feature_engineering.FeatureEngineer.calculate_time_features (
    self,
    pd.DataFrame df)
```

Calculate time-based features with safety checks

7.3.2.10 calculate_volatility_features()

```
pd.DataFrame feature_engineering.FeatureEngineer.calculate_volatility_features (
    self,
    pd.DataFrame df)
```

Calculate volatility features

7.3.2.11 calculate_volume_features()

```
pd.DataFrame feature_engineering.FeatureEngineer.calculate_volume_features (
    self,
    pd.DataFrame df)
```

Calculate volume-based features

7.3.2.12 create_feature_matrix()

```
pd.DataFrame feature_engineering.FeatureEngineer.create_feature_matrix (
    self,
    pd.DataFrame df)
```

Create complete feature matrix

7.3.2.13 get_feature_columns()

```
List[str] feature_engineering.FeatureEngineer.get_feature_columns (
    self)
```

Get list of feature columns

7.3.2.14 normalize_features()

```
pd.DataFrame feature_engineering.FeatureEngineer.normalize_features (
    self,
    pd.DataFrame df)
```

Normalize features using Z-score normalization

7.3.3 Member Data Documentation**7.3.3.1 computation_time_limit**

```
int feature_engineering.FeatureEngineer.computation_time_limit = 5
```

7.3.3.2 error_count

```
int feature_engineering.FeatureEngineer.error_count = 0
```

7.3.3.3 feature_blacklist

```
feature_engineering.FeatureEngineer.feature_blacklist = set(self.user_config.get('blacklisted←
_features', []))
```

7.3.3.4 feature_cache

```
dict feature_engineering.FeatureEngineer.feature_cache = {}
```

7.3.3.5 feature_calculation_times

```
dict feature_engineering.FeatureEngineer.feature_calculation_times = {}
```

7.3.3.6 feature_columns

```
feature_engineering.FeatureEngineer.feature_columns = df.columns.tolist()
```

7.3.3.7 feature_importance

```
dict feature_engineering.FeatureEngineer.feature_importance = {}
```

7.3.3.8 feature_importance_threshold

```
float feature_engineering.FeatureEngineer.feature_importance_threshold = 0.01
```

7.3.3.9 feature_selector

```
feature_engineering.FeatureEngineer.feature_selector = None
```

7.3.3.10 feature_whitelist

```
feature_engineering.FeatureEngineer.feature_whitelist = set(self.user_config.get('whitelisted←  
_features', []))
```

7.3.3.11 last_feature_selection

```
feature_engineering.FeatureEngineer.last_feature_selection = None
```

7.3.3.12 last_feature_update

```
feature_engineering.FeatureEngineer.last_feature_update = datetime.now()
```

7.3.3.13 last_update

```
feature_engineering.FeatureEngineer.last_update = None
```

7.3.3.14 logger

```
feature_engineering.FeatureEngineer.logger = logging.getLogger(__name__)
```

7.3.3.15 max_errors

```
int feature_engineering.FeatureEngineer.max_errors = 5
```

7.3.3.16 max_features

```
int feature_engineering.FeatureEngineer.max_features = 100
```

7.3.3.17 memory_limit

```
int feature_engineering.FeatureEngineer.memory_limit = 1e9
```

7.3.3.18 memory_usage

```
dict feature_engineering.FeatureEngineer.memory_usage = {}
```

7.3.3.19 momentum_features

```
feature_engineering.FeatureEngineer.momentum_features = momentum_features
```

7.3.3.20 scaler

```
feature_engineering.FeatureEngineer.scaler = StandardScaler()
```

7.3.3.21 technical_indicators

```
feature_engineering.FeatureEngineer.technical_indicators = technical_indicators
```

7.3.3.22 user_config

```
feature_engineering.FeatureEngineer.user_config = user_config or {}
```

7.3.3.23 user_override

```
bool feature_engineering.FeatureEngineer.user_override = False
```

7.3.3.24 volatility_features

```
feature_engineering.FeatureEngineer.volatility_features = volatility_features
```

7.3.3.25 volume_features

```
feature_engineering.FeatureEngineer.volume_features = volume_features
```

7.3.3.26 window_sizes

```
feature_engineering.FeatureEngineer.window_sizes = window_sizes
```

The documentation for this class was generated from the following file:

- [backend/ai_models/feature_engineering.py](#)

7.4 legal.LegalProtector Class Reference

Public Member Functions

- [__init__](#) (self)
- bool [show_disclaimer](#) (self)
- bool [require_user_agreement](#) (self)
- Dict[str, Any] [validate_usage](#) (self)
- None [log_action](#) (self, str action_type, Dict[str, Any] details)
- str [get_disclaimer](#) (self)
- str [get_user_agreement](#) (self)
- str [get_privacy_policy](#) (self)
- Dict[str, Any] [check_compliance](#) (self)

Public Attributes

- [logger](#) = logging.getLogger(__name__)
- bool [disclaimer_shown](#) = False
- bool [user_agreement_accepted](#) = False
- [last_disclaimer_check](#) = None
- [last_agreement_check](#) = None
- str [disclaimer](#) = self._load_legal_document('disclaimer.txt')
- str [user_agreement](#) = self._load_legal_document('user_agreement.txt')
- str [privacy_policy](#) = self._load_legal_document('privacy_policy.txt')
- str [compliance_dir](#) = Path(__file__).parent.parent / 'compliance'

Protected Member Functions

- str [_load_legal_document](#) (self, str filename)
- None [_save_compliance_record](#) (self, str record_type, Dict[str, Any] data)

7.4.1 Detailed Description

Class for handling legal compliance and user protection

7.4.2 Constructor & Destructor Documentation

7.4.2.1 __init__()

```
legal.LegalProtector.__init__ (
    self)
```

Initialize legal protection system

7.4.3 Member Function Documentation

7.4.3.1 `_load_legal_document()`

```
str legal.LegalProtector._load_legal_document (  
    self,  
    str filename) [protected]
```

Load legal document from file

7.4.3.2 `_save_compliance_record()`

```
None legal.LegalProtector._save_compliance_record (  
    self,  
    str record_type,  
    Dict[str, Any] data) [protected]
```

Save compliance record to file

7.4.3.3 `check_compliance()`

```
Dict[str, Any] legal.LegalProtector.check_compliance (  
    self)
```

Check overall compliance status

7.4.3.4 `get_disclaimer()`

```
str legal.LegalProtector.get_disclaimer (  
    self)
```

Get disclaimer text

7.4.3.5 `get_privacy_policy()`

```
str legal.LegalProtector.get_privacy_policy (  
    self)
```

Get privacy policy text

7.4.3.6 `get_user_agreement()`

```
str legal.LegalProtector.get_user_agreement (  
    self)
```

Get user agreement text

7.4.3.7 `log_action()`

```
None legal.LegalProtector.log_action (  
    self,  
    str action_type,  
    Dict[str, Any] details)
```

Log user actions for compliance

7.4.3.8 require_user_agreement()

```
bool legal.LegalProtector.require_user_agreement (  
    self)
```

Require user to accept terms of service

7.4.3.9 show_disclaimer()

```
bool legal.LegalProtector.show_disclaimer (  
    self)
```

Show disclaimer to user and get acceptance

7.4.3.10 validate_usage()

```
Dict[str, Any] legal.LegalProtector.validate_usage (  
    self)
```

Validate legal compliance before usage

7.4.4 Member Data Documentation

7.4.4.1 compliance_dir

```
str legal.LegalProtector.compliance_dir = Path(__file__).parent.parent / 'compliance'
```

7.4.4.2 disclaimer

```
str legal.LegalProtector.disclaimer = self._load_legal_document('disclaimer.txt')
```

7.4.4.3 disclaimer_shown

```
legal.LegalProtector.disclaimer_shown = False
```

7.4.4.4 last_agreement_check

```
legal.LegalProtector.last_agreement_check = None
```

7.4.4.5 last_disclaimer_check

```
legal.LegalProtector.last_disclaimer_check = None
```

7.4.4.6 logger

```
legal.LegalProtector.logger = logging.getLogger(__name__)
```

7.4.4.7 privacy_policy

```
str legal.LegalProtector.privacy_policy = self._load_legal_document('privacy_policy.txt')
```

7.4.4.8 user_agreement

```
str legal.LegalProtector.user_agreement = self._load_legal_document('user_agreement.txt')
```

7.4.4.9 user_agreement_accepted

```
bool legal.LegalProtector.user_agreement_accepted = False
```

The documentation for this class was generated from the following file:

- backend/utils/[legal.py](#)

7.5 lstm_model.LSTMMModel Class Reference

Public Member Functions

- `__init__` (self, Tuple[int, int] `input_shape`=(60, 10), float `dropout_rate`=0.2, float `learning_rate`=0.001, int `batch_size`=32, int `epochs`=100, int `patience`=10, int `verbose`=1, Dict `user_config`=None)
- `build_model` (self)
- None `train` (self, np.ndarray `X_train`, np.ndarray `y_train`, Optional[np.ndarray] `X_val`=None, Optional[np.ndarray] `y_val`=None)
- np.ndarray `predict` (self, np.ndarray `X`)
- Tuple[np.ndarray, np.ndarray] `prepare_data` (self, pd.DataFrame `data`)
- None `train` (self, np.ndarray `X_train`, np.ndarray `y_train`, Optional[np.ndarray] `X_val`=None, Optional[np.ndarray] `y_val`=None)
- np.ndarray `predict` (self, np.ndarray `X`)
- Dict[str, float] `evaluate` (self, np.ndarray `X_test`, np.ndarray `y_test`)
- None `save_model` (self, str `path`)
- None `load_model` (self, str `path`)
- Dict[str, List[float]] `get_metrics` (self)
- str `get_model_summary` (self)

Public Attributes

- `input_shape` = input_shape
- `dropout_rate` = dropout_rate
- `learning_rate` = learning_rate
- `batch_size` = batch_size
- `epochs` = epochs
- `patience` = patience
- `verbose` = verbose
- None `model` = None
- `scaler` = StandardScaler()
- dict `metrics`
- `last_update` = None
- int `max_memory_usage` = 2e9
- int `max_training_time` = 3600
- int `max_prediction_time` = 5
- float `error_threshold` = 0.05
- int `max_consecutive_errors` = 3
- int `error_count` = 0
- `last_error` = None
- `user_config` = user_config or {}
- bool `user_override` = False
- bool `user_paused` = False
- bool `user_configured` = False
- list `training_times` = []
- list `prediction_times` = []
- list `memory_usage` = []
- list `cpu_usage` = []
- bool `is_training` = False
- bool `is_predicting` = False
- `last_training_start` = None
- `last_prediction_start` = None
- `last_memory_check` = None
- `logger` = logging.getLogger(__name__)
- dict `train_metrics`
- `last_error_timestamp` = datetime.now()
- dict `model_config`
- `last_checkpoint` = datetime.now()

Protected Member Functions

- [_validate_configuration](#) (self)
- [bool _check_memory_usage](#) (self)
- [bool _check_cpu_usage](#) (self)
- [_log_model_state](#) (self, str state)
- [_handle_error](#) (self, Exception error, str operation)
- [bool _validate_data](#) (self, np.ndarray X, Optional[np.ndarray] y=None)
- [_save_metrics](#) (self)

7.5.1 Constructor & Destructor Documentation

7.5.1.1 __init__()

```
lstm_model.LSTMModel.__init__ (
    self,
    Tuple[int, int]  input_shape = (60, 10),
    float  dropout_rate = 0.2,
    float  learning_rate = 0.001,
    int  batch_size = 32,
    int  epochs = 100,
    int  patience = 10,
    int  verbose = 1,
    Dict  user_config = None)
```

Initialize LSTM model with safety measures

Args:

input_shape: Shape of input data (timesteps, features)
 dropout_rate: Dropout rate for regularization
 learning_rate: Learning rate for optimizer
 batch_size: Batch size for training
 epochs: Number of training epochs
 patience: Patience for early stopping
 verbose: Verbosity level
 user_config: User-defined configuration overrides

7.5.2 Member Function Documentation

7.5.2.1 _check_cpu_usage()

```
bool lstm_model.LSTMModel._check_cpu_usage (
    self) [protected]
```

Check if CPU usage is within limits

7.5.2.2 _check_memory_usage()

```
bool lstm_model.LSTMModel._check_memory_usage (
    self) [protected]
```

Check if memory usage is within limits

7.5.2.3 _handle_error()

```
lstm_model.LSTMModel._handle_error (
    self,
    Exception error,
    str operation) [protected]
```

Handle model errors with enhanced safety measures

7.5.2.4 `_log_model_state()`

```
lstm_model.LSTMModel._log_model_state (
    self,
    str state) [protected]
```

Log model state changes

7.5.2.5 `_save_metrics()`

```
lstm_model.LSTMModel._save_metrics (
    self) [protected]
```

Save comprehensive metrics with enhanced monitoring

7.5.2.6 `_validate_configuration()`

```
lstm_model.LSTMModel._validate_configuration (
    self) [protected]
```

Validate user configuration

7.5.2.7 `_validate_data()`

```
bool lstm_model.LSTMModel._validate_data (
    self,
    np.ndarray X,
    Optional[np.ndarray] y = None) [protected]
```

Validate input data with enhanced checks

7.5.2.8 `build_model()`

```
lstm_model.LSTMModel.build_model (
    self)
```

Build enhanced LSTM model with safety measures

7.5.2.9 `evaluate()`

```
Dict[str, float] lstm_model.LSTMModel.evaluate (
    self,
    np.ndarray X_test,
    np.ndarray y_test)
```

Evaluate model performance

7.5.2.10 `get_metrics()`

```
Dict[str, List[float]] lstm_model.LSTMModel.get_metrics (
    self)
```

Get training metrics history

7.5.2.11 `get_model_summary()`

```
str lstm_model.LSTMModel.get_model_summary (
    self)
```

Get model architecture summary

7.5.2.12 load_model()

```
None lstm_model.LSTMModel.load_model (  
    self,  
    str path)
```

Load a pre-trained model

7.5.2.13 predict() [1/2]

```
np.ndarray lstm_model.LSTMModel.predict (  
    self,  
    np.ndarray X)
```

Make predictions with enhanced safety and monitoring

7.5.2.14 predict() [2/2]

```
np.ndarray lstm_model.LSTMModel.predict (  
    self,  
    np.ndarray X)
```

Make predictions using the trained model

7.5.2.15 prepare_data()

```
Tuple[np.ndarray, np.ndarray] lstm_model.LSTMModel.prepare_data (  
    self,  
    pd.DataFrame data)
```

Prepare data for LSTM input

7.5.2.16 save_model()

```
None lstm_model.LSTMModel.save_model (  
    self,  
    str path)
```

Save the trained model

7.5.2.17 train() [1/2]

```
None lstm_model.LSTMModel.train (  
    self,  
    np.ndarray X_train,  
    np.ndarray y_train,  
    Optional[np.ndarray] X_val = None,  
    Optional[np.ndarray] y_val = None)
```

Train LSTM model with enhanced safety and monitoring

7.5.2.18 train() [2/2]

```
None lstm_model.LSTMModel.train (  
    self,  
    np.ndarray X_train,  
    np.ndarray y_train,  
    Optional[np.ndarray] X_val = None,  
    Optional[np.ndarray] y_val = None)
```

Train the LSTM model

7.5.3 Member Data Documentation

7.5.3.1 batch_size

```
lstm_model.LSTMModel.batch_size = batch_size
```

7.5.3.2 cpu_usage

```
list lstm_model.LSTMModel.cpu_usage = []
```

7.5.3.3 dropout_rate

```
lstm_model.LSTMModel.dropout_rate = dropout_rate
```

7.5.3.4 epochs

```
lstm_model.LSTMModel.epochs = epochs
```

7.5.3.5 error_count

```
int lstm_model.LSTMModel.error_count = 0
```

7.5.3.6 error_threshold

```
float lstm_model.LSTMModel.error_threshold = 0.05
```

7.5.3.7 input_shape

```
lstm_model.LSTMModel.input_shape = input_shape
```

7.5.3.8 is_predicting

```
bool lstm_model.LSTMModel.is_predicting = False
```

7.5.3.9 is_training

```
bool lstm_model.LSTMModel.is_training = False
```

7.5.3.10 last_checkpoint

```
lstm_model.LSTMModel.last_checkpoint = datetime.now()
```

7.5.3.11 last_error

```
lstm_model.LSTMModel.last_error = None
```

7.5.3.12 last_error_timestamp

```
lstm_model.LSTMModel.last_error_timestamp = datetime.now()
```

7.5.3.13 last_memory_check

```
lstm_model.LSTMModel.last_memory_check = None
```

7.5.3.14 last_prediction_start

```
lstm_model.LSTMModel.last_prediction_start = None
```

7.5.3.15 last_training_start

```
lstm_model.LSTMModel.last_training_start = None
```

7.5.3.16 last_update

```
lstm_model.LSTMModel.last_update = None
```

7.5.3.17 learning_rate

```
lstm_model.LSTMModel.learning_rate = learning_rate
```

7.5.3.18 logger

```
lstm_model.LSTMModel.logger = logging.getLogger(__name__)
```

7.5.3.19 max_consecutive_errors

```
int lstm_model.LSTMModel.max_consecutive_errors = 3
```

7.5.3.20 max_memory_usage

```
int lstm_model.LSTMModel.max_memory_usage = 2e9
```

7.5.3.21 max_prediction_time

```
int lstm_model.LSTMModel.max_prediction_time = 5
```

7.5.3.22 max_training_time

```
int lstm_model.LSTMModel.max_training_time = 3600
```

7.5.3.23 memory_usage

```
list lstm_model.LSTMModel.memory_usage = []
```

7.5.3.24 metrics

```
dict lstm_model.LSTMModel.metrics
```

Initial value:

```
= {
    'loss': [],
    'val_loss': [],
    'accuracy': [],
    'val_accuracy': []
}
```

7.5.3.25 model

```
None lstm_model.LSTMModel.model = None
```

7.5.3.26 model_config

```
dict lstm_model.LSTMModel.model_config
```

Initial value:

```
= {
    'architecture': model.to_json(),
    'optimizer': str(optimizer)
}
```

7.5.3.27 patience

```
lstm_model.LSTMModel.patience = patience
```

7.5.3.28 prediction_times

```
list lstm_model.LSTMModel.prediction_times = []
```

7.5.3.29 scaler

```
lstm_model.LSTMModel.scaler = StandardScaler()
```

7.5.3.30 train_metrics

```
dict lstm_model.LSTMModel.train_metrics
```

Initial value:

```
= {
    'loss': [],
    'accuracy': [],
    'val_loss': [],
    'val_accuracy': [],
    'memory_usage': [],
    'cpu_usage': [],
    'training_time': [],
    'prediction_time': []
}
```

7.5.3.31 training_times

```
list lstm_model.LSTMModel.training_times = []
```

7.5.3.32 user_config

```
lstm_model.LSTMModel.user_config = user_config or {}
```

7.5.3.33 user_configured

```
bool lstm_model.LSTMModel.user_configured = False
```

7.5.3.34 user_override

```
bool lstm_model.LSTMModel.user_override = False
```

7.5.3.35 user_paused

```
bool lstm_model.LSTMModel.user_paused = False
```

7.5.3.36 verbose

```
lstm_model.LSTMModel.verbose = verbose
```

The documentation for this class was generated from the following file:

- [backend/ai_models/lstm_model.py](#)

7.6 market_predictor.MarketPredictor Class Reference**Public Member Functions**

- `__init__` (self, int [prediction_horizon](#)=1, int [lookback_window](#)=60, str [model_type](#)='random_forest', float [feature_importance_threshold](#)=0.01, float [validation_split](#)=0.2)
- Tuple[np.ndarray, np.ndarray] [prepare_features](#) (self, pd.DataFrame df)
- None [train_model](#) (self, np.ndarray X, np.ndarray y)
- np.ndarray [predict](#) (self, np.ndarray X)
- Dict[str, float] [evaluate](#) (self, np.ndarray X, np.ndarray y)
- Dict[str, float] [get_feature_importances](#) (self)
- Dict[str, float] [get_model_metrics](#) (self)
- pd.DataFrame [create_prediction_pipeline](#) (self, pd.DataFrame df)

Public Attributes

- `prediction_horizon` = `prediction_horizon`
- `lookback_window` = `lookback_window`
- `str model_type` = `model_type`
- `feature_importance_threshold` = `feature_importance_threshold`
- `validation_split` = `validation_split`
- `model` = `None`
- `scaler` = `StandardScaler()`
- `dict feature_importances` = `{}`
- `Optional[datetime] last_update` = `None`

7.6.1 Constructor & Destructor Documentation

7.6.1.1 `__init__()`

```
market_predictor.MarketPredictor.__init__ (
    self,
    int prediction_horizon = 1,
    int lookback_window = 60,
    str model_type = 'random_forest',
    float feature_importance_threshold = 0.01,
    float validation_split = 0.2)
```

Initialize market predictor

Args:

```
prediction_horizon: Number of periods to predict
lookback_window: Number of past periods to use
model_type: Type of prediction model
feature_importance_threshold: Minimum importance for features
validation_split: Split ratio for validation set
```

7.6.2 Member Function Documentation

7.6.2.1 `create_prediction_pipeline()`

```
pd.DataFrame market_predictor.MarketPredictor.create_prediction_pipeline (
    self,
    pd.DataFrame df)
```

Create complete prediction pipeline

7.6.2.2 `evaluate()`

```
Dict[str, float] market_predictor.MarketPredictor.evaluate (
    self,
    np.ndarray X,
    np.ndarray y)
```

Evaluate model performance

7.6.2.3 `get_feature_importances()`

```
Dict[str, float] market_predictor.MarketPredictor.get_feature_importances (
    self)
```

Get feature importances

7.6.2.4 get_model_metrics()

```
Dict[str, float] market_predictor.MarketPredictor.get_model_metrics (  
    self)
```

Get model metrics

7.6.2.5 predict()

```
np.ndarray market_predictor.MarketPredictor.predict (  
    self,  
    np.ndarray X)
```

Make predictions

7.6.2.6 prepare_features()

```
Tuple[np.ndarray, np.ndarray] market_predictor.MarketPredictor.prepare_features (  
    self,  
    pd.DataFrame df)
```

Prepare features and target for prediction

7.6.2.7 train_model()

```
None market_predictor.MarketPredictor.train_model (  
    self,  
    np.ndarray X,  
    np.ndarray y)
```

Train prediction model

7.6.3 Member Data Documentation

7.6.3.1 feature_importance_threshold

```
market_predictor.MarketPredictor.feature_importance_threshold = feature_importance_threshold
```

7.6.3.2 feature_importances

```
market_predictor.MarketPredictor.feature_importances = {}
```

7.6.3.3 last_update

```
Optional[datetime] market_predictor.MarketPredictor.last_update = None
```

7.6.3.4 lookback_window

```
market_predictor.MarketPredictor.lookback_window = lookback_window
```

7.6.3.5 model

```
market_predictor.MarketPredictor.model = None
```

7.6.3.6 model_type

```
market_predictor.MarketPredictor.model_type = model_type
```

7.6.3.7 prediction_horizon

```
market_predictor.MarketPredictor.prediction_horizon = prediction_horizon
```

7.6.3.8 scaler

```
market_predictor.MarketPredictor.scaler = StandardScaler()
```

7.6.3.9 validation_split

```
market_predictor.MarketPredictor.validation_split = validation_split
```

The documentation for this class was generated from the following file:

- [backend/ai_models/market_predictor.py](#)

7.7 market_simulation.MarketSimulator Class Reference

Public Member Functions

- `__init__` (self, pd.DataFrame [market_data](#), int [order_book_depth](#)=10, float [liquidity_threshold](#)=0.05, float [market_impact_exponent](#)=0.6, float [slippage_base](#)=0.0005)
- Dict [simulate_order_book](#) (self, float price, float volume, str side)
- float [calculate_market_impact](#) (self, float trade_size, float market_volume)
- Dict [simulate_trade](#) (self, float price, float volume, str side, float market_volume)
- Dict [simulate_market_impact](#) (self, List[Dict] trades, float market_volume)
- Dict [simulate_liquidity](#) (self, List[Dict] trades, float market_volume)
- Dict [simulate_trading_day](#) (self, List[Dict] trades, float market_volume)
- None [plot_market_simulation](#) (self, List[Dict] trades, float market_volume)
- Dict [get_simulation_metrics](#) (self)

Public Attributes

- [market_data](#) = market_data
- [order_book_depth](#) = order_book_depth
- [liquidity_threshold](#) = liquidity_threshold
- [market_impact_exponent](#) = market_impact_exponent
- [slippage_base](#) = slippage_base
- [metrics](#) = Metrics()
- [cost_analyzer](#) = cost_analyzer

7.7.1 Constructor & Destructor Documentation

7.7.1.1 __init__()

```
market_simulation.MarketSimulator.__init__ (
    self,
    pd.DataFrame market_data,
    int order_book_depth = 10,
    float liquidity_threshold = 0.05,
    float market_impact_exponent = 0.6,
    float slippage_base = 0.0005)
```

Initialize market simulator

Args:

```
market_data: Market data DataFrame
order_book_depth: Depth of order book to simulate
liquidity_threshold: Threshold for liquidity impact
market_impact_exponent: Exponent for market impact calculation
slippage_base: Base slippage percentage
```

7.7.2 Member Function Documentation

7.7.2.1 calculate_market_impact()

```
float market_simulation.MarketSimulator.calculate_market_impact (  
    self,  
    float trade_size,  
    float market_volume)
```

Calculate market impact

7.7.2.2 get_simulation_metrics()

```
Dict market_simulation.MarketSimulator.get_simulation_metrics (  
    self)
```

Get simulation parameters

7.7.2.3 plot_market_simulation()

```
None market_simulation.MarketSimulator.plot_market_simulation (  
    self,  
    List[Dict] trades,  
    float market_volume)
```

Plot market simulation results

7.7.2.4 simulate_liquidity()

```
Dict market_simulation.MarketSimulator.simulate_liquidity (  
    self,  
    List[Dict] trades,  
    float market_volume)
```

Simulate liquidity impact

7.7.2.5 simulate_market_impact()

```
Dict market_simulation.MarketSimulator.simulate_market_impact (  
    self,  
    List[Dict] trades,  
    float market_volume)
```

Simulate market impact of multiple trades

7.7.2.6 simulate_order_book()

```
Dict market_simulation.MarketSimulator.simulate_order_book (  
    self,  
    float price,  
    float volume,  
    str side)
```

Simulate order book execution

7.7.2.7 simulate_trade()

```
Dict market_simulation.MarketSimulator.simulate_trade (
    self,
    float price,
    float volume,
    str side,
    float market_volume)
```

Simulate trade execution

7.7.2.8 simulate_trading_day()

```
Dict market_simulation.MarketSimulator.simulate_trading_day (
    self,
    List[Dict] trades,
    float market_volume)
```

Simulate a full trading day

7.7.3 Member Data Documentation

7.7.3.1 cost_analyzer

```
market_simulation.MarketSimulator.cost_analyzer = cost_analyzer
```

7.7.3.2 liquidity_threshold

```
market_simulation.MarketSimulator.liquidity_threshold = liquidity_threshold
```

7.7.3.3 market_data

```
market_simulation.MarketSimulator.market_data = market_data
```

7.7.3.4 market_impact_exponent

```
market_simulation.MarketSimulator.market_impact_exponent = market_impact_exponent
```

7.7.3.5 metrics

```
market_simulation.MarketSimulator.metrics = Metrics()
```

7.7.3.6 order_book_depth

```
market_simulation.MarketSimulator.order_book_depth = order_book_depth
```

7.7.3.7 slippage_base

```
market_simulation.MarketSimulator.slippage_base = slippage_base
```

The documentation for this class was generated from the following file:

- backend/[market_simulation.py](#)

7.8 meta_learner.MetaLearner Class Reference

Public Member Functions

- `__init__` (self, str [history_file](#)='data/trade_history.json', int [lookback_window](#)=30, float [confidence_threshold](#)=0.↵6, int [min_samples](#)=10)

- List[Dict] [load_history](#) (self)
- Dict[str, float] [evaluate_strategies](#) (self, List[Dict] history)
- bool [train_model](#) (self, bool force=False)
- List[Tuple[str, float]] [recommend_strategies](#) (self)
- float [confidence_score](#) (self, str strategy)
- Dict [get_strategy_metrics](#) (self, str strategy)

Public Attributes

- [history_file](#) = history_file
- dict [strategy_scores](#) = {}
- [lookback_window](#) = lookback_window
- [confidence_threshold](#) = confidence_threshold
- [min_samples](#) = min_samples
- [model](#) = RandomForestRegressor(n_estimators=100, random_state=42)
- [scaler](#) = StandardScaler()
- list [feature_columns](#) = ['pnl', 'volatility', 'market_trend', 'time_of_day']
- Optional[datetime] [last_update](#) = None

Protected Member Functions

- Dict [_calculate_features](#) (self, Dict trade)
- float [_get_time_of_day_feature](#) (self, str timestamp)
- Tuple[np.ndarray, np.ndarray] [_prepare_features](#) (self, List[Dict] history)

7.8.1 Constructor & Destructor Documentation

7.8.1.1 __init__()

```
meta_learner.MetaLearner.__init__ (
    self,
    str history_file = 'data/trade_history.json',
    int lookback_window = 30,
    float confidence_threshold = 0.6,
    int min_samples = 10)
```

7.8.2 Member Function Documentation

7.8.2.1 _calculate_features()

```
Dict meta_learner.MetaLearner._calculate_features (
    self,
    Dict trade) [protected]
```

Calculate advanced features for each trade

7.8.2.2 _get_time_of_day_feature()

```
float meta_learner.MetaLearner._get_time_of_day_feature (
    self,
    str timestamp) [protected]
```

Convert timestamp to time of day feature

7.8.2.3 `_prepare_features()`

```
Tuple[np.ndarray, np.ndarray] meta_learner.MetaLearner._prepare_features (  
    self,  
    List[Dict] history) [protected]
```

Prepare features and labels for training

7.8.2.4 `confidence_score()`

```
float meta_learner.MetaLearner.confidence_score (  
    self,  
    str strategy)
```

Calculate confidence score using multiple metrics

7.8.2.5 `evaluate_strategies()`

```
Dict[str, float] meta_learner.MetaLearner.evaluate_strategies (  
    self,  
    List[Dict] history)
```

Evaluate strategies using advanced statistical methods

7.8.2.6 `get_strategy_metrics()`

```
Dict meta_learner.MetaLearner.get_strategy_metrics (  
    self,  
    str strategy)
```

Get detailed metrics for a specific strategy

7.8.2.7 `load_history()`

```
List[Dict] meta_learner.MetaLearner.load_history (  
    self)
```

Load and validate trade history

7.8.2.8 `recommend_strategies()`

```
List[Tuple[str, float]] meta_learner.MetaLearner.recommend_strategies (  
    self)
```

Recommend strategies with confidence scores

7.8.2.9 `train_model()`

```
bool meta_learner.MetaLearner.train_model (  
    self,  
    bool force = False)
```

Train the meta-learning model

7.8.3 Member Data Documentation

7.8.3.1 `confidence_threshold`

```
meta_learner.MetaLearner.confidence_threshold = confidence_threshold
```

7.8.3.2 feature_columns

```
list meta_learner.MetaLearner.feature_columns = ['pnl', 'volatility', 'market_trend', 'time_of_day']
```

7.8.3.3 history_file

```
meta_learner.MetaLearner.history_file = history_file
```

7.8.3.4 last_update

```
meta_learner.MetaLearner.last_update = None
```

7.8.3.5 lookback_window

```
meta_learner.MetaLearner.lookback_window = lookback_window
```

7.8.3.6 min_samples

```
meta_learner.MetaLearner.min_samples = min_samples
```

7.8.3.7 model

```
meta_learner.MetaLearner.model = RandomForestRegressor(n_estimators=100, random_state=42)
```

7.8.3.8 scaler

```
meta_learner.MetaLearner.scaler = StandardScaler()
```

7.8.3.9 strategy_scores

```
dict meta_learner.MetaLearner.strategy_scores = {}
```

The documentation for this class was generated from the following file:

- [backend/ai_models/meta_learner.py](#)

7.9 meta_reinforcement.MetaReinforcementLearner Class Reference

Public Member Functions

- [__init__](#) (self)
- [record_trade_result](#) (self, strategy, reward)
- [choose_strategy](#) (self)

Public Attributes

- [rewards](#) = defaultdict(list)
- [q_scores](#) = defaultdict(float)

7.9.1 Constructor & Destructor Documentation

7.9.1.1 __init__()

```
meta_reinforcement.MetaReinforcementLearner.__init__ (
    self)
```

7.9.2 Member Function Documentation

7.9.2.1 choose_strategy()

```
meta_reinforcement.MetaReinforcementLearner.choose_strategy (
    self)
```

7.9.2.2 record_trade_result()

```
meta_reinforcement.MetaReinforcementLearner.record_trade_result (
    self,
    strategy,
    reward)
```

7.9.3 Member Data Documentation

7.9.3.1 q_scores

```
meta_reinforcement.MetaReinforcementLearner.q_scores = defaultdict(float)
```

7.9.3.2 rewards

```
meta_reinforcement.MetaReinforcementLearner.rewards = defaultdict(list)
```

The documentation for this class was generated from the following file:

- [backend/ai_models/meta_reinforcement.py](#)

7.10 metrics.Metrics Class Reference

Public Member Functions

- `__init__` (self, float [risk_free_rate](#)=0.02, Optional[pd.Series] [benchmark_returns](#)=None, int [window_size](#)=252, bool [cache_enabled](#)=True)
- float [calculate_sharpe_ratio](#) (self, pd.Series returns)
- float [calculate_max_drawdown](#) (self, pd.Series returns)
- Dict [calculate_basic_metrics](#) (self, pd.Series returns)

Public Attributes

- [risk_free_rate](#) = risk_free_rate
- [benchmark_returns](#) = benchmark_returns
- [window_size](#) = window_size
- [cache_enabled](#) = cache_enabled
- dict [metrics_cache](#) = {}

Protected Member Functions

- str [_get_cache_key](#) (self, pd.Series returns, str method_name)
- None [_cache_result](#) (self, str key, Any result)
- Optional[Any] [_get_cached_result](#) (self, str key)

7.10.1 Constructor & Destructor Documentation

7.10.1.1 __init__()

```
metrics.Metrics.__init__ (
    self,
    float risk_free_rate = 0.02,
    Optional[pd.Series] benchmark_returns = None,
```



```
int    window_size = 252,
bool   cache_enabled = True)
```

Initialize metrics calculator

Args:

```
risk_free_rate: Annual risk-free rate
benchmark_returns: Benchmark returns series
window_size: Rolling window size
cache_enabled: Enable caching of calculations
```

7.10.2 Member Function Documentation

7.10.2.1 `_cache_result()`

```
None metrics.Metrics._cache_result (
    self,
    str key,
    Any result) [protected]
```

Cache result with TTL

7.10.2.2 `_get_cache_key()`

```
str metrics.Metrics._get_cache_key (
    self,
    pd.Series returns,
    str method_name) [protected]
```

Generate cache key based on returns and method

7.10.2.3 `_get_cached_result()`

```
Optional[Any] metrics.Metrics._get_cached_result (
    self,
    str key) [protected]
```

Get cached result if valid

7.10.2.4 `calculate_basic_metrics()`

```
Dict metrics.Metrics.calculate_basic_metrics (
    self,
    pd.Series returns)
```

Calculate basic performance metrics with caching and optimization

7.10.2.5 `calculate_max_drawdown()`

```
float metrics.Metrics.calculate_max_drawdown (
    self,
    pd.Series returns)
```

Calculate maximum drawdown with caching and optimization

7.10.2.6 `calculate_sharpe_ratio()`

```
float metrics.Metrics.calculate_sharpe_ratio (
    self,
    pd.Series returns)
```

Calculate Sharpe ratio with caching and optimization

7.10.3 Member Data Documentation

7.10.3.1 benchmark_returns

```
metrics.Metrics.benchmark_returns = benchmark_returns
```

7.10.3.2 cache_enabled

```
metrics.Metrics.cache_enabled = cache_enabled
```

7.10.3.3 metrics_cache

```
dict metrics.Metrics.metrics_cache = {}
```

7.10.3.4 risk_free_rate

```
metrics.Metrics.risk_free_rate = risk_free_rate
```

7.10.3.5 window_size

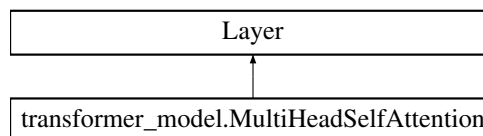
```
metrics.Metrics.window_size = window_size
```

The documentation for this class was generated from the following file:

- backend/[metrics.py](#)

7.11 transformer_model.MultiHeadSelfAttention Class Reference

Inheritance diagram for transformer_model.MultiHeadSelfAttention:



Public Member Functions

- [__init__](#) (self, [embed_dim](#), [num_heads](#)=8)
- [attention](#) (self, query, key, value)
- [separate_heads](#) (self, x, batch_size)
- [call](#) (self, inputs)

Public Attributes

- [embed_dim](#) = [embed_dim](#)
- [num_heads](#) = [num_heads](#)
- [projection_dim](#) = [embed_dim](#) // [num_heads](#)
- [query_dense](#) = [Dense](#)([embed_dim](#))
- [key_dense](#) = [Dense](#)([embed_dim](#))
- [value_dense](#) = [Dense](#)([embed_dim](#))
- [combine_heads](#) = [Dense](#)([embed_dim](#))

7.11.1 Constructor & Destructor Documentation

7.11.1.1 __init__()

```
transformer_model.MultiHeadSelfAttention.__init__ (
    self,
    embed_dim,
    num_heads = 8)
```

7.11.2 Member Function Documentation

7.11.2.1 attention()

```
transformer_model.MultiHeadSelfAttention.attention (  
    self,  
    query,  
    key,  
    value)
```

7.11.2.2 call()

```
transformer_model.MultiHeadSelfAttention.call (  
    self,  
    inputs)
```

7.11.2.3 separate_heads()

```
transformer_model.MultiHeadSelfAttention.separate_heads (  
    self,  
    x,  
    batch_size)
```

7.11.3 Member Data Documentation

7.11.3.1 combine_heads

```
transformer_model.MultiHeadSelfAttention.combine_heads = Dense(embed_dim)
```

7.11.3.2 embed_dim

```
transformer_model.MultiHeadSelfAttention.embed_dim = embed_dim
```

7.11.3.3 key_dense

```
transformer_model.MultiHeadSelfAttention.key_dense = Dense(embed_dim)
```

7.11.3.4 num_heads

```
transformer_model.MultiHeadSelfAttention.num_heads = num_heads
```

7.11.3.5 projection_dim

```
transformer_model.MultiHeadSelfAttention.projection_dim = embed_dim // num_heads
```

7.11.3.6 query_dense

```
transformer_model.MultiHeadSelfAttention.query_dense = Dense(embed_dim)
```

7.11.3.7 value_dense

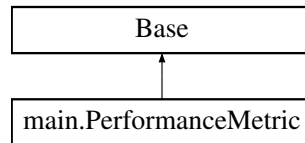
```
transformer_model.MultiHeadSelfAttention.value_dense = Dense(embed_dim)
```

The documentation for this class was generated from the following file:

- backend/ai_models/[transformer_model.py](#)

7.12 main.PerformanceMetric Class Reference

Inheritance diagram for main.PerformanceMetric:



Static Public Attributes

- `id` = Column(Integer, primary_key=True, index=True)
- `strategy` = Column(String, index=True)
- `metric_name` = Column(String, index=True)
- `value` = Column(Float)
- `timestamp` = Column(DateTime)

7.12.1 Member Data Documentation

7.12.1.1 id

```
main.PerformanceMetric.id = Column(Integer, primary_key=True, index=True) [static]
```

7.12.1.2 metric_name

```
main.PerformanceMetric.metric_name = Column(String, index=True) [static]
```

7.12.1.3 strategy

```
main.PerformanceMetric.strategy = Column(String, index=True) [static]
```

7.12.1.4 timestamp

```
main.PerformanceMetric.timestamp = Column(DateTime) [static]
```

7.12.1.5 value

```
main.PerformanceMetric.value = Column(Float) [static]
```

The documentation for this class was generated from the following file:

- backend/[main.py](#)

7.13 main.PerformanceMonitor Class Reference

Public Member Functions

- `__init__` (self)
- `__call__` (self, Request request, call_next)

Public Attributes

- list `requests` = []

7.13.1 Constructor & Destructor Documentation

7.13.1.1 __init__()

```
main.PerformanceMonitor.__init__ (
    self)
```

7.13.2 Member Function Documentation

7.13.2.1 __call__()

```
main.PerformanceMonitor.__call__ (
    self,
    Request request,
    call_next)
```

7.13.3 Member Data Documentation

7.13.3.1 requests

```
list main.PerformanceMonitor.requests = []
```

The documentation for this class was generated from the following file:

- backend/[main.py](#)

7.14 portfolio_optimizer.PortfolioOptimizer Class Reference

Public Member Functions

- [__init__](#) (self, int [max_assets](#)=10, float [min_allocation](#)=0.01, float [max_allocation](#)=0.3, float [risk_free_rate](#)=0.02, str [optimization_method](#)='sharpe_ratio')
- pd.DataFrame [calculate_covariance_matrix](#) (self, pd.DataFrame returns)
- float [calculate_portfolio_return](#) (self, np.ndarray weights, pd.DataFrame returns)
- float [calculate_portfolio_volatility](#) (self, np.ndarray weights, pd.DataFrame cov_matrix)
- float [calculate_sharpe_ratio](#) (self, np.ndarray weights, pd.DataFrame returns, pd.DataFrame cov_matrix)
- Dict[str, float] [optimize_portfolio](#) (self, pd.DataFrame returns, Optional[pd.DataFrame] cov_matrix=None)
- pd.DataFrame [calculate_efficient_frontier](#) (self, pd.DataFrame returns, int num_points=20)
- Dict[str, float] [rebalance_portfolio](#) (self, Dict[str, float] [current_weights](#), Dict[str, float] [target_weights](#), float [transaction_cost](#)=0.001)
- [__init__](#) (self, float [risk_free_rate](#)=0.02, float [max_leverage](#)=2.0, float [min_allocation](#)=0.01, float [max_allocation](#)=0.3, int [rebalance_period](#)=252, str [optimization_method](#)='risk_parity', Dict [constraints](#)=None)
- pd.DataFrame [calculate_covariance_matrix](#) (self, pd.DataFrame returns)
- pd.DataFrame [calculate_correlation_matrix](#) (self, pd.DataFrame returns)
- float [calculate_volatility](#) (self, pd.Series returns)
- np.ndarray [calculate_risk_parity_weights](#) (self, np.ndarray volatilities, np.ndarray correlations)
- np.ndarray [calculate_min_variance_weights](#) (self, np.ndarray cov_matrix)
- np.ndarray [calculate_max_diversification_weights](#) (self, np.ndarray volatilities, np.ndarray correlations)
- Dict[str, float] [optimize_portfolio](#) (self, pd.DataFrame returns)
- Dict[str, float] [rebalance_portfolio](#) (self, Dict[str, float] [current_weights](#), Dict[str, float] [target_weights](#), Dict[str, float] [prices](#), float [portfolio_value](#))
- Dict [calculate_portfolio_metrics](#) (self, pd.DataFrame returns, Dict[str, float] [weights](#))
- Dict [get_optimization_metrics](#) (self)
- None [plot_allocation](#) (self, Dict[str, float] [weights](#))

Public Attributes

- `max_assets` = `max_assets`
- `min_allocation` = `min_allocation`
- `max_allocation` = `max_allocation`
- `risk_free_rate` = `risk_free_rate`
- `str optimization_method` = `optimization_method`
- `tuple constraints`
- `max_leverage` = `max_leverage`
- `rebalance_period` = `rebalance_period`
- `metrics` = `Metrics(risk_free_rate)`
- `last_rebalance` = `None`
- `dict current_weights` = `{}`

7.14.1 Constructor & Destructor Documentation**7.14.1.1 `__init__()` [1/2]**

```
portfolio_optimizer.PortfolioOptimizer.__init__ (
    self,
    int    max_assets = 10,
    float  min_allocation = 0.01,
    float  max_allocation = 0.3,
    float  risk_free_rate = 0.02,
    str    optimization_method = 'sharpe_ratio')
```

Initialize portfolio optimizer

Args:

```
max_assets: Maximum number of assets in portfolio
min_allocation: Minimum allocation per asset
max_allocation: Maximum allocation per asset
risk_free_rate: Risk-free rate for Sharpe ratio calculation
optimization_method: Method for portfolio optimization
```

7.14.1.2 `__init__()` [2/2]

```
portfolio_optimizer.PortfolioOptimizer.__init__ (
    self,
    float  risk_free_rate = 0.02,
    float  max_leverage = 2.0,
    float  min_allocation = 0.01,
    float  max_allocation = 0.3,
    int    rebalance_period = 252,
    str    optimization_method = 'risk_parity',
    Dict   constraints = None)
```

Initialize portfolio optimizer

Args:

```
risk_free_rate: Annual risk-free rate
max_leverage: Maximum leverage allowed
min_allocation: Minimum allocation per asset
max_allocation: Maximum allocation per asset
rebalance_period: Period for rebalancing
optimization_method: Portfolio optimization method
constraints: Additional optimization constraints
```

7.14.2 Member Function Documentation

7.14.2.1 calculate_correlation_matrix()

```
pd.DataFrame portfolio_optimizer.PortfolioOptimizer.calculate_correlation_matrix (  
    self,  
    pd.DataFrame returns)
```

Calculate correlation matrix

7.14.2.2 calculate_covariance_matrix() [1/2]

```
pd.DataFrame portfolio_optimizer.PortfolioOptimizer.calculate_covariance_matrix (  
    self,  
    pd.DataFrame returns)
```

Calculate covariance matrix of returns

7.14.2.3 calculate_covariance_matrix() [2/2]

```
pd.DataFrame portfolio_optimizer.PortfolioOptimizer.calculate_covariance_matrix (  
    self,  
    pd.DataFrame returns)
```

Calculate covariance matrix

7.14.2.4 calculate_efficient_frontier()

```
pd.DataFrame portfolio_optimizer.PortfolioOptimizer.calculate_efficient_frontier (  
    self,  
    pd.DataFrame returns,  
    int num_points = 20)
```

Calculate efficient frontier

7.14.2.5 calculate_max_diversification_weights()

```
np.ndarray portfolio_optimizer.PortfolioOptimizer.calculate_max_diversification_weights (  
    self,  
    np.ndarray volatilities,  
    np.ndarray correlations)
```

Calculate maximum diversification weights

7.14.2.6 calculate_min_variance_weights()

```
np.ndarray portfolio_optimizer.PortfolioOptimizer.calculate_min_variance_weights (  
    self,  
    np.ndarray cov_matrix)
```

Calculate minimum variance weights

7.14.2.7 calculate_portfolio_metrics()

```
Dict portfolio_optimizer.PortfolioOptimizer.calculate_portfolio_metrics (  
    self,  
    pd.DataFrame returns,  
    Dict[str, float] weights)
```

Calculate portfolio metrics

7.14.2.8 calculate_portfolio_return()

```
float portfolio_optimizer.PortfolioOptimizer.calculate_portfolio_return (  
    self,  
    np.ndarray weights,  
    pd.DataFrame returns)
```

Calculate portfolio return

7.14.2.9 calculate_portfolio_volatility()

```
float portfolio_optimizer.PortfolioOptimizer.calculate_portfolio_volatility (  
    self,  
    np.ndarray weights,  
    pd.DataFrame cov_matrix)
```

Calculate portfolio volatility

7.14.2.10 calculate_risk_parity_weights()

```
np.ndarray portfolio_optimizer.PortfolioOptimizer.calculate_risk_parity_weights (  
    self,  
    np.ndarray volatilities,  
    np.ndarray correlations)
```

Calculate risk parity weights

7.14.2.11 calculate_sharpe_ratio()

```
float portfolio_optimizer.PortfolioOptimizer.calculate_sharpe_ratio (  
    self,  
    np.ndarray weights,  
    pd.DataFrame returns,  
    pd.DataFrame cov_matrix)
```

Calculate Sharpe ratio

7.14.2.12 calculate_volatility()

```
float portfolio_optimizer.PortfolioOptimizer.calculate_volatility (  
    self,  
    pd.Series returns)
```

Calculate annualized volatility

7.14.2.13 get_optimization_metrics()

```
Dict portfolio_optimizer.PortfolioOptimizer.get_optimization_metrics (  
    self)
```

Get optimization parameters and metrics

7.14.2.14 optimize_portfolio() [1/2]

```
Dict[str, float] portfolio_optimizer.PortfolioOptimizer.optimize_portfolio (  
    self,  
    pd.DataFrame returns)
```

Optimize portfolio weights

7.14.2.15 optimize_portfolio() [2/2]

```
Dict[str, float] portfolio_optimizer.PortfolioOptimizer.optimize_portfolio (
    self,
    pd.DataFrame returns,
    Optional[pd.DataFrame] cov_matrix = None)
```

Optimize portfolio weights

7.14.2.16 plot_allocation()

```
None portfolio_optimizer.PortfolioOptimizer.plot_allocation (
    self,
    Dict[str, float] weights)
```

Plot portfolio allocation

7.14.2.17 rebalance_portfolio() [1/2]

```
Dict[str, float] portfolio_optimizer.PortfolioOptimizer.rebalance_portfolio (
    self,
    Dict[str, float] current_weights,
    Dict[str, float] target_weights,
    Dict[str, float] prices,
    float portfolio_value)
```

Calculate rebalancing trades

7.14.2.18 rebalance_portfolio() [2/2]

```
Dict[str, float] portfolio_optimizer.PortfolioOptimizer.rebalance_portfolio (
    self,
    Dict[str, float] current_weights,
    Dict[str, float] target_weights,
    float transaction_cost = 0.001)
```

Calculate portfolio rebalancing

7.14.3 Member Data Documentation**7.14.3.1 constraints**

portfolio_optimizer.PortfolioOptimizer.constraints

Initial value:

```
= (
    {'type': 'eq', 'fun': lambda x: np.sum(x) - 1}, # Sum of weights = 1
    {'type': 'ineq', 'fun': lambda x: x - self.min_allocation}, # Min allocation
    {'type': 'ineq', 'fun': lambda x: self.max_allocation - x} # Max allocation
)
```

7.14.3.2 current_weights

dict portfolio_optimizer.PortfolioOptimizer.current_weights = {}

7.14.3.3 last_rebalance

portfolio_optimizer.PortfolioOptimizer.last_rebalance = None

7.14.3.4 max_allocation

portfolio_optimizer.PortfolioOptimizer.max_allocation = max_allocation

7.14.3.5 max_assets

```
portfolio_optimizer.PortfolioOptimizer.max_assets = max_assets
```

7.14.3.6 max_leverage

```
portfolio_optimizer.PortfolioOptimizer.max_leverage = max_leverage
```

7.14.3.7 metrics

```
portfolio_optimizer.PortfolioOptimizer.metrics = Metrics(risk_free_rate)
```

7.14.3.8 min_allocation

```
portfolio_optimizer.PortfolioOptimizer.min_allocation = min_allocation
```

7.14.3.9 optimization_method

```
str portfolio_optimizer.PortfolioOptimizer.optimization_method = optimization_method
```

7.14.3.10 rebalance_period

```
portfolio_optimizer.PortfolioOptimizer.rebalance_period = rebalance_period
```

7.14.3.11 risk_free_rate

```
portfolio_optimizer.PortfolioOptimizer.risk_free_rate = risk_free_rate
```

The documentation for this class was generated from the following files:

- backend/ai_models/[portfolio_optimizer.py](#)
- backend/[portfolio_optimizer.py](#)

7.15 main.RateLimiter Class Reference

Public Member Functions

- [__init__](#) (self, int [max_requests](#)=100, int [window_seconds](#)=60)
- [__call__](#) (self, Request request, call_next)

Public Attributes

- [max_requests](#) = max_requests
- [window_seconds](#) = window_seconds
- dict [requests](#) = {}

7.15.1 Constructor & Destructor Documentation

7.15.1.1 __init__()

```
main.RateLimiter.__init__ (
    self,
    int  max_requests = 100,
    int  window_seconds = 60)
```

7.15.2 Member Function Documentation

7.15.2.1 `__call__()`

```
main.RateLimiter.__call__ (
    self,
    Request request,
    call_next)
```

7.15.3 Member Data Documentation

7.15.3.1 `max_requests`

```
main.RateLimiter.max_requests = max_requests
```

7.15.3.2 `requests`

```
dict main.RateLimiter.requests = {}
```

7.15.3.3 `window_seconds`

```
main.RateLimiter.window_seconds = window_seconds
```

The documentation for this class was generated from the following file:

- backend/[main.py](#)

7.16 real_time_analysis.RealTimeAnalyzer Class Reference

Public Member Functions

- `__init__` (self, int `window_size`=252, int `update_frequency`=60, float `risk_threshold`=0.02, float `rebalance_threshold`=0.05, float `max_position_size`=0.05)
- None `update_positions` (self, Dict[str, float] positions)
- Dict[str, bool] `check_risk_limits` (self)
- float `calculate_position_sizing` (self, float price, float volatility, float portfolio_value)
- Dict[str, float] `optimize_positions` (self, Dict[str, float] `current_positions`, Dict[str, float] `target_positions`, float `portfolio_value`)
- Dict `get_real_time_metrics` (self)
- None `plot_real_time_analysis` (self)
- Dict[str, float] `get_position_adjustments` (self, Dict[str, float] `current_positions`, pd.DataFrame `price_data`)

Public Attributes

- `window_size` = window_size
- `update_frequency` = update_frequency
- `risk_threshold` = risk_threshold
- `rebalance_threshold` = rebalance_threshold
- `max_position_size` = max_position_size
- `metrics` = Metrics()
- `last_update` = None
- dict `current_positions` = {}
- dict `risk_metrics` = {}

Protected Member Functions

- None `_calculate_risk_metrics` (self)

7.16.1 Constructor & Destructor Documentation

7.16.1.1 `__init__()`

```
real_time_analysis.RealTimeAnalyzer.__init__ (
    self,
    int    window_size = 252,
    int    update_frequency = 60,
    float   risk_threshold = 0.02,
    float   rebalance_threshold = 0.05,
    float   max_position_size = 0.05)
```

Initialize real-time analyzer

Args:

```
window_size: Rolling window size
update_frequency: Frequency of updates (seconds)
risk_threshold: Risk threshold for alerts
rebalance_threshold: Threshold for rebalancing
max_position_size: Maximum position size
```

7.16.2 Member Function Documentation

7.16.2.1 `_calculate_risk_metrics()`

```
None real_time_analysis.RealTimeAnalyzer._calculate_risk_metrics (
    self) [protected]
```

Calculate risk metrics

7.16.2.2 `calculate_position_sizing()`

```
float real_time_analysis.RealTimeAnalyzer.calculate_position_sizing (
    self,
    float price,
    float volatility,
    float portfolio_value)
```

Calculate optimal position size

7.16.2.3 `check_risk_limits()`

```
Dict[str, bool] real_time_analysis.RealTimeAnalyzer.check_risk_limits (
    self)
```

Check if risk limits are exceeded

7.16.2.4 `get_position_adjustments()`

```
Dict[str, float] real_time_analysis.RealTimeAnalyzer.get_position_adjustments (
    self,
    Dict[str, float] current_positions,
    pd.DataFrame price_data)
```

Calculate position adjustments

7.16.2.5 `get_real_time_metrics()`

```
Dict real_time_analysis.RealTimeAnalyzer.get_real_time_metrics (
    self)
```

Get current real-time metrics

7.16.2.6 optimize_positions()

```
Dict[str, float] real_time_analysis.RealTimeAnalyzer.optimize_positions (
    self,
    Dict[str, float] current_positions,
    Dict[str, float] target_positions,
    float portfolio_value)
```

Optimize positions based on risk and cost

7.16.2.7 plot_real_time_analysis()

```
None real_time_analysis.RealTimeAnalyzer.plot_real_time_analysis (
    self)
```

Plot real-time analysis results

7.16.2.8 update_positions()

```
None real_time_analysis.RealTimeAnalyzer.update_positions (
    self,
    Dict[str, float] positions)
```

Update current positions

7.16.3 Member Data Documentation

7.16.3.1 current_positions

```
real_time_analysis.RealTimeAnalyzer.current_positions = {}
```

7.16.3.2 last_update

```
real_time_analysis.RealTimeAnalyzer.last_update = None
```

7.16.3.3 max_position_size

```
real_time_analysis.RealTimeAnalyzer.max_position_size = max_position_size
```

7.16.3.4 metrics

```
real_time_analysis.RealTimeAnalyzer.metrics = Metrics()
```

7.16.3.5 rebalance_threshold

```
real_time_analysis.RealTimeAnalyzer.rebalance_threshold = rebalance_threshold
```

7.16.3.6 risk_metrics

```
real_time_analysis.RealTimeAnalyzer.risk_metrics = {}
```

7.16.3.7 risk_threshold

```
real_time_analysis.RealTimeAnalyzer.risk_threshold = risk_threshold
```

7.16.3.8 update_frequency

```
real_time_analysis.RealTimeAnalyzer.update_frequency = update_frequency
```

7.16.3.9 window_size

`real_time_analysis.RealTimeAnalyzer.window_size = window_size`

The documentation for this class was generated from the following file:

- [backend/real_time_analysis.py](#)

7.17 main.RequestValidator Class Reference

Public Member Functions

- [__init__](#) (self)
- [validate_request](#) (self, Request request)

Public Attributes

- [executor](#) = ThreadPoolExecutor(max_workers=4)

Protected Member Functions

- [_validate_ip](#) (self, Request request)
- [_validate_rate_limit](#) (self, Request request)
- [_validate_headers](#) (self, Request request)

7.17.1 Constructor & Destructor Documentation

7.17.1.1 __init__()

```
main.RequestValidator.__init__ (
    self)
```

7.17.2 Member Function Documentation

7.17.2.1 _validate_headers()

```
main.RequestValidator._validate_headers (
    self,
    Request request) [protected]
```

7.17.2.2 _validate_ip()

```
main.RequestValidator._validate_ip (
    self,
    Request request) [protected]
```

7.17.2.3 _validate_rate_limit()

```
main.RequestValidator._validate_rate_limit (
    self,
    Request request) [protected]
```

7.17.2.4 validate_request()

```
main.RequestValidator.validate_request (
    self,
    Request request)
```

7.17.3 Member Data Documentation

7.17.3.1 executor

`main.RequestValidator.executor = ThreadPoolExecutor(max_workers=4)`

The documentation for this class was generated from the following file:

- [backend/main.py](#)

7.18 risk_management.RiskManager Class Reference

Public Member Functions

- `__init__` (self, float [max_position_size](#)=0.05, float [max_drawdown](#)=0.02, float [stop_loss_pct](#)=0.02, float [take_profit_pct](#)=0.03, float [risk_per_trade](#)=0.01, bool [volatility_adjustment](#)=True, float [correlation_threshold](#)=0.8, float [max_leverage](#)=2.0)
- float [calculate_position_size](#) (self, float current_price, float portfolio_value, float volatility, float leverage=1.0)
- Tuple[float, float] [calculate_stop_loss](#) (self, float entry_price, float volatility, float position_size)
- float [calculate_risk_reward_ratio](#) (self, float stop_loss, float take_profit, float entry_price)
- None [update_metrics](#) (self, float trade_pnl, float portfolio_value)
- Dict[str, float] [get_risk_metrics](#) (self)
- bool [check_position_correlation](#) (self, str new_position)
- None [update_correlation_matrix](#) (self, pd.DataFrame price_data)
- Dict[str, float] [get_position_limits](#) (self, float portfolio_value)

Public Attributes

- [max_position_size](#) = max_position_size
- [max_drawdown](#) = max_drawdown
- [stop_loss_pct](#) = stop_loss_pct
- [take_profit_pct](#) = take_profit_pct
- [risk_per_trade](#) = risk_per_trade
- [volatility_adjustment](#) = volatility_adjustment
- [correlation_threshold](#) = correlation_threshold
- [max_leverage](#) = max_leverage
- float [current_drawdown](#) = 0.0
- float [max_drawdown_reached](#) = 0.0
- int [total_trades](#) = 0
- int [winning_trades](#) = 0
- int [losing_trades](#) = 0
- float [total_profit](#) = 0.0
- dict [current_positions](#) = {}
- [correlation_matrix](#) = None

7.18.1 Constructor & Destructor Documentation

7.18.1.1 __init__()

```
risk_management.RiskManager.__init__ (
    self,
    float max_position_size = 0.05,
    float max_drawdown = 0.02,
    float stop_loss_pct = 0.02,
    float take_profit_pct = 0.03,
    float risk_per_trade = 0.01,
    bool volatility_adjustment = True,
    float correlation_threshold = 0.8,
    float max_leverage = 2.0)
```

Initialize risk management system

Args:

```

max_position_size: Maximum position size as % of portfolio
max_drawdown: Maximum allowed drawdown
stop_loss_pct: Stop loss percentage
take_profit_pct: Take profit percentage
risk_per_trade: Risk per trade as % of portfolio
volatility_adjustment: Whether to adjust position size based on volatility
correlation_threshold: Maximum allowed correlation between positions
max_leverage: Maximum leverage allowed

```

7.18.2 Member Function Documentation

7.18.2.1 calculate_position_size()

```

float risk_management.RiskManager.calculate_position_size (
    self,
    float current_price,
    float portfolio_value,
    float volatility,
    float leverage = 1.0)

```

Calculate optimal position size

7.18.2.2 calculate_risk_reward_ratio()

```

float risk_management.RiskManager.calculate_risk_reward_ratio (
    self,
    float stop_loss,
    float take_profit,
    float entry_price)

```

Calculate risk/reward ratio

7.18.2.3 calculate_stop_loss()

```

Tuple[float, float] risk_management.RiskManager.calculate_stop_loss (
    self,
    float entry_price,
    float volatility,
    float position_size)

```

Calculate stop loss and take profit levels

7.18.2.4 check_position_correlation()

```

bool risk_management.RiskManager.check_position_correlation (
    self,
    str new_position)

```

Check correlation with existing positions

7.18.2.5 get_position_limits()

```

Dict[str, float] risk_management.RiskManager.get_position_limits (
    self,
    float portfolio_value)

```

Get position limits based on current portfolio

7.18.2.6 get_risk_metrics()

```
Dict[str, float] risk_management.RiskManager.get_risk_metrics (  
    self)
```

Get current risk metrics

7.18.2.7 update_correlation_matrix()

```
None risk_management.RiskManager.update_correlation_matrix (  
    self,  
    pd.DataFrame price_data)
```

Update correlation matrix

7.18.2.8 update_metrics()

```
None risk_management.RiskManager.update_metrics (  
    self,  
    float trade_pnl,  
    float portfolio_value)
```

Update risk metrics

7.18.3 Member Data Documentation

7.18.3.1 correlation_matrix

```
risk_management.RiskManager.correlation_matrix = None
```

7.18.3.2 correlation_threshold

```
risk_management.RiskManager.correlation_threshold = correlation_threshold
```

7.18.3.3 current_drawdown

```
risk_management.RiskManager.current_drawdown = 0.0
```

7.18.3.4 current_positions

```
dict risk_management.RiskManager.current_positions = {}
```

7.18.3.5 losing_trades

```
risk_management.RiskManager.losing_trades = 0
```

7.18.3.6 max_drawdown

```
risk_management.RiskManager.max_drawdown = max_drawdown
```

7.18.3.7 max_drawdown_reached

```
risk_management.RiskManager.max_drawdown_reached = 0.0
```

7.18.3.8 max_leverage

```
risk_management.RiskManager.max_leverage = max_leverage
```

7.18.3.9 max_position_size

```
risk_management.RiskManager.max_position_size = max_position_size
```

7.18.3.10 risk_per_trade

```
risk_management.RiskManager.risk_per_trade = risk_per_trade
```

7.18.3.11 stop_loss_pct

```
risk_management.RiskManager.stop_loss_pct = stop_loss_pct
```

7.18.3.12 take_profit_pct

```
risk_management.RiskManager.take_profit_pct = take_profit_pct
```

7.18.3.13 total_profit

```
risk_management.RiskManager.total_profit = 0.0
```

7.18.3.14 total_trades

```
risk_management.RiskManager.total_trades = 0
```

7.18.3.15 volatility_adjustment

```
risk_management.RiskManager.volatility_adjustment = volatility_adjustment
```

7.18.3.16 winning_trades

```
risk_management.RiskManager.winning_trades = 0
```

The documentation for this class was generated from the following file:

- [backend/risk_management.py](#)

7.19 risk_manager.RiskManager Class Reference**Public Member Functions**

- `__init__` (self, float [max_position_size](#)=0.05, float [max_drawdown](#)=0.02, float [stop_loss_pct](#)=0.02, float [take_profit_pct](#)=0.03, float [risk_per_trade](#)=0.01, bool [volatility_adjustment](#)=True)
- float [calculate_position_size](#) (self, float current_price, float portfolio_value, float volatility)
- Tuple[float, float] [calculate_stop_loss](#) (self, float entry_price, float volatility)
- float [calculate_risk_reward_ratio](#) (self, float stop_loss, float take_profit, float entry_price)
- None [update_metrics](#) (self, float trade_pnl, float portfolio_value)
- Dict[str, float] [get_risk_metrics](#) (self)
- bool [check_risk_limits](#) (self, float portfolio_value, float position_size)
- float [calculate_hedging_position](#) (self, float current_position, str market_trend, float volatility)

Public Attributes

- [max_position_size](#) = max_position_size
- [max_drawdown](#) = max_drawdown
- [stop_loss_pct](#) = stop_loss_pct
- [take_profit_pct](#) = take_profit_pct
- [risk_per_trade](#) = risk_per_trade
- [volatility_adjustment](#) = volatility_adjustment
- float [current_drawdown](#) = 0.0
- float [max_drawdown_reached](#) = 0.0

- int `total_trades` = 0
- int `winning_trades` = 0
- int `losing_trades` = 0
- float `total_profit` = 0.0

7.19.1 Constructor & Destructor Documentation

7.19.1.1 `__init__()`

```
risk_manager.RiskManager.__init__ (
    self,
    float max_position_size = 0.05,
    float max_drawdown = 0.02,
    float stop_loss_pct = 0.02,
    float take_profit_pct = 0.03,
    float risk_per_trade = 0.01,
    bool volatility_adjustment = True)
```

Initialize risk management system

Args:

```
max_position_size: Maximum position size as % of portfolio
max_drawdown: Maximum allowed drawdown
stop_loss_pct: Stop loss percentage
take_profit_pct: Take profit percentage
risk_per_trade: Risk per trade as % of portfolio
volatility_adjustment: Whether to adjust position size based on volatility
```

7.19.2 Member Function Documentation

7.19.2.1 `calculate_hedging_position()`

```
float risk_manager.RiskManager.calculate_hedging_position (
    self,
    float current_position,
    str market_trend,
    float volatility)
```

Calculate optimal hedging position

7.19.2.2 `calculate_position_size()`

```
float risk_manager.RiskManager.calculate_position_size (
    self,
    float current_price,
    float portfolio_value,
    float volatility)
```

Calculate optimal position size

7.19.2.3 `calculate_risk_reward_ratio()`

```
float risk_manager.RiskManager.calculate_risk_reward_ratio (
    self,
    float stop_loss,
    float take_profit,
    float entry_price)
```

Calculate risk/reward ratio

7.19.2.4 calculate_stop_loss()

```
Tuple[float, float] risk_manager.RiskManager.calculate_stop_loss (  
    self,  
    float entry_price,  
    float volatility)
```

Calculate stop loss and take profit levels

7.19.2.5 check_risk_limits()

```
bool risk_manager.RiskManager.check_risk_limits (  
    self,  
    float portfolio_value,  
    float position_size)
```

Check if risk limits are exceeded

7.19.2.6 get_risk_metrics()

```
Dict[str, float] risk_manager.RiskManager.get_risk_metrics (  
    self)
```

Get current risk metrics

7.19.2.7 update_metrics()

```
None risk_manager.RiskManager.update_metrics (  
    self,  
    float trade_pnl,  
    float portfolio_value)
```

Update risk metrics

7.19.3 Member Data Documentation

7.19.3.1 current_drawdown

```
risk_manager.RiskManager.current_drawdown = 0.0
```

7.19.3.2 losing_trades

```
risk_manager.RiskManager.losing_trades = 0
```

7.19.3.3 max_drawdown

```
risk_manager.RiskManager.max_drawdown = max_drawdown
```

7.19.3.4 max_drawdown_reached

```
risk_manager.RiskManager.max_drawdown_reached = 0.0
```

7.19.3.5 max_position_size

```
risk_manager.RiskManager.max_position_size = max_position_size
```

7.19.3.6 risk_per_trade

```
risk_manager.RiskManager.risk_per_trade = risk_per_trade
```

7.19.3.7 stop_loss_pct

```
risk_manager.RiskManager.stop_loss_pct = stop_loss_pct
```

7.19.3.8 take_profit_pct

```
risk_manager.RiskManager.take_profit_pct = take_profit_pct
```

7.19.3.9 total_profit

```
float risk_manager.RiskManager.total_profit = 0.0
```

7.19.3.10 total_trades

```
risk_manager.RiskManager.total_trades = 0
```

7.19.3.11 volatility_adjustment

```
risk_manager.RiskManager.volatility_adjustment = volatility_adjustment
```

7.19.3.12 winning_trades

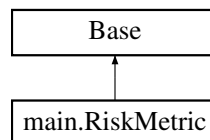
```
risk_manager.RiskManager.winning_trades = 0
```

The documentation for this class was generated from the following file:

- backend/ai_models/[risk_manager.py](#)

7.20 main.RiskMetric Class Reference

Inheritance diagram for main.RiskMetric:

**Static Public Attributes**

- [id](#) = Column(Integer, primary_key=True, index=True)
- [symbol](#) = Column(String, index=True)
- [metric_name](#) = Column(String, index=True)
- [value](#) = Column(Float)
- [timestamp](#) = Column(DateTime)

7.20.1 Member Data Documentation**7.20.1.1 id**

```
main.RiskMetric.id = Column(Integer, primary_key=True, index=True) [static]
```

7.20.1.2 metric_name

```
main.RiskMetric.metric_name = Column(String, index=True) [static]
```

7.20.1.3 symbol

```
main.RiskMetric.symbol = Column(String, index=True) [static]
```

7.20.1.4 timestamp

```
main.RiskMetric.timestamp = Column(DateTime) [static]
```

7.20.1.5 value

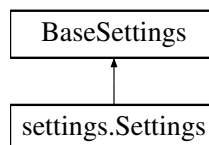
```
main.RiskMetric.value = Column(Float) [static]
```

The documentation for this class was generated from the following file:

- backend/[main.py](#)

7.21 settings.Settings Class Reference

Inheritance diagram for settings.Settings:



Static Public Attributes

- bool [trading_enabled](#) = True
- str [currency_mode](#) = "multi"
- bool [logging_enabled](#) = True
- str [twelve_data_key](#) = "YOUR_TWELVE_DATA_API_KEY"
- str [telegram_token](#) = "YOUR_TELEGRAM_BOT_TOKEN"
- str [telegram_chat_id](#) = "YOUR_TELEGRAM_CHAT_ID"

7.21.1 Member Data Documentation

7.21.1.1 currency_mode

```
str settings.Settings.currency_mode = "multi" [static]
```

7.21.1.2 logging_enabled

```
bool settings.Settings.logging_enabled = True [static]
```

7.21.1.3 telegram_chat_id

```
str settings.Settings.telegram_chat_id = "YOUR_TELEGRAM_CHAT_ID" [static]
```

7.21.1.4 telegram_token

```
str settings.Settings.telegram_token = "YOUR_TELEGRAM_BOT_TOKEN" [static]
```

7.21.1.5 trading_enabled

```
bool settings.Settings.trading_enabled = True [static]
```

7.21.1.6 twelve_data_key

```
str settings.Settings.twelve_data_key = "YOUR_TWELVE_DATA_API_KEY" [static]
```

The documentation for this class was generated from the following file:

- backend/config/[settings.py](#)

7.22 strategy_framework.StrategyFramework Class Reference

Public Member Functions

- `__init__` (self, List[Type[TradingStrategy]] strategies, Dict[str, float] weights=None, RiskManager risk_manager=None, Backtester backtester=None)
- bool `execute_trade` (self, dict trade_details)
- None `initialize_strategies` (self, pd.DataFrame data)
- None `update_metrics` (self, dict trade_result)
- bool `validate_weights` (self)
- Dict[str, float] `optimize_weights` (self)
- pd.Series `generate_combined_signals` (self, pd.DataFrame data)
- None `rebalance` (self, pd.DataFrame data)
- Dict[str, Dict] `get_strategy_metrics` (self)
- Dict `get_combined_metrics` (self, pd.DataFrame data)
- None `plot_strategy_performance` (self)

Public Attributes

- strategies = strategies
- dict weights = weights or {s.__name__: 1/len(strategies) for s in strategies}
- risk_manager = risk_manager or RiskManager()
- backtester = backtester or Backtester()
- dict strategy_instances = {}
- last_update = None
- float max_position_size = 0.1
- float max_drawdown = 0.05
- int consecutive_losses = 0
- int current_drawdown = 0
- bool active = True
- correlation_threshold
- last_rebalance
- correlation_matrix

Protected Member Functions

- bool `_validate_weights` (self)
- bool `_check_safety` (self, dict trade_details)

7.22.1 Constructor & Destructor Documentation

7.22.1.1 __init__()

```
strategy_framework.StrategyFramework.__init__ (
    self,
    List[Type[TradingStrategy]] strategies,
    Dict[str, float] weights = None,
    RiskManager risk_manager = None,
    Backtester backtester = None)
```

Initialize strategy framework with safety measures

Args:

```
strategies: List of strategy classes
weights: Weights for each strategy
risk_manager: Risk management instance
backtester: Backtesting instance
```

7.22.2 Member Function Documentation

7.22.2.1 `_check_safety()`

```
bool strategy_framework.StrategyFramework._check_safety (  
    self,  
    dict trade_details) [protected]
```

Check if trade is within safety parameters

7.22.2.2 `_validate_weights()`

```
bool strategy_framework.StrategyFramework._validate_weights (  
    self) [protected]
```

Validate strategy weights

7.22.2.3 `execute_trade()`

```
bool strategy_framework.StrategyFramework.execute_trade (  
    self,  
    dict trade_details)
```

Execute trade with safety checks

7.22.2.4 `generate_combined_signals()`

```
pd.Series strategy_framework.StrategyFramework.generate_combined_signals (  
    self,  
    pd.DataFrame data)
```

Generate combined signals from all strategies

7.22.2.5 `get_combined_metrics()`

```
Dict strategy_framework.StrategyFramework.get_combined_metrics (  
    self,  
    pd.DataFrame data)
```

Get combined strategy metrics

7.22.2.6 `get_strategy_metrics()`

```
Dict[str, Dict] strategy_framework.StrategyFramework.get_strategy_metrics (  
    self)
```

Get metrics for all strategies

7.22.2.7 `initialize_strategies()`

```
None strategy_framework.StrategyFramework.initialize_strategies (  
    self,  
    pd.DataFrame data)
```

Initialize all strategies with safety checks

7.22.2.8 optimize_weights()

```
Dict[str, float] strategy_framework.StrategyFramework.optimize_weights (  
    self)
```

Optimize strategy weights based on correlations and performance

7.22.2.9 plot_strategy_performance()

```
None strategy_framework.StrategyFramework.plot_strategy_performance (  
    self)
```

Plot strategy performance

7.22.2.10 rebalance()

```
None strategy_framework.StrategyFramework.rebalance (  
    self,  
    pd.DataFrame data)
```

Rebalance strategy weights

7.22.2.11 update_metrics()

```
None strategy_framework.StrategyFramework.update_metrics (  
    self,  
    dict trade_result)
```

Update safety metrics based on trade results

7.22.2.12 validate_weights()

```
bool strategy_framework.StrategyFramework.validate_weights (  
    self)
```

Validate that weights sum to 1

7.22.3 Member Data Documentation

7.22.3.1 active

```
bool strategy_framework.StrategyFramework.active = True
```

7.22.3.2 backtester

```
strategy_framework.StrategyFramework.backtester = backtester or Backtester()
```

7.22.3.3 consecutive_losses

```
strategy_framework.StrategyFramework.consecutive_losses = 0
```

7.22.3.4 correlation_matrix

```
strategy_framework.StrategyFramework.correlation_matrix
```

7.22.3.5 correlation_threshold

```
strategy_framework.StrategyFramework.correlation_threshold
```

7.22.3.6 current_drawdown

```
strategy_framework.StrategyFramework.current_drawdown = 0
```

7.22.3.7 last_rebalance

```
strategy_framework.StrategyFramework.last_rebalance
```

7.22.3.8 last_update

```
strategy_framework.StrategyFramework.last_update = None
```

7.22.3.9 max_drawdown

```
float strategy_framework.StrategyFramework.max_drawdown = 0.05
```

7.22.3.10 max_position_size

```
float strategy_framework.StrategyFramework.max_position_size = 0.1
```

7.22.3.11 risk_manager

```
strategy_framework.StrategyFramework.risk_manager = risk_manager or RiskManager()
```

7.22.3.12 strategies

```
strategy_framework.StrategyFramework.strategies = strategies
```

7.22.3.13 strategy_instances

```
dict strategy_framework.StrategyFramework.strategy_instances = {}
```

7.22.3.14 weights

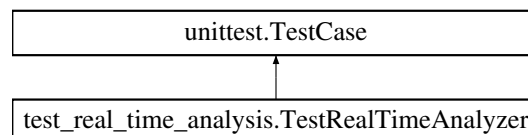
```
strategy_framework.StrategyFramework.weights = weights or {s.__name__: 1/len(strategies) for s in strategies}
```

The documentation for this class was generated from the following file:

- [backend/strategies/strategy_framework.py](#)

7.23 test_real_time_analysis.TestRealTimeAnalyzer Class Reference

Inheritance diagram for test_real_time_analysis.TestRealTimeAnalyzer:

**Public Member Functions**

- [setUp](#) (self)
- [test_update_positions](#) (self)
- [test_check_risk_limits](#) (self)
- [test_calculate_position_sizing](#) (self)
- [test_optimize_positions](#) (self)
- [test_get_real_time_metrics](#) (self)
- [test_plot_real_time_analysis](#) (self)
- [test_get_position_adjustments](#) (self)

Public Attributes

- [analyzer](#)
- dict [positions](#)
- [returns](#)
- [price_data](#)

7.23.1 Member Function Documentation**7.23.1.1 setUp()**

```
test_real_time_analysis.TestRealTimeAnalyzer.setUp (
    self)
```

7.23.1.2 test_calculate_position_sizing()

```
test_real_time_analysis.TestRealTimeAnalyzer.test_calculate_position_sizing (
    self)
```

7.23.1.3 test_check_risk_limits()

```
test_real_time_analysis.TestRealTimeAnalyzer.test_check_risk_limits (
    self)
```

7.23.1.4 test_get_position_adjustments()

```
test_real_time_analysis.TestRealTimeAnalyzer.test_get_position_adjustments (
    self)
```

7.23.1.5 test_get_real_time_metrics()

```
test_real_time_analysis.TestRealTimeAnalyzer.test_get_real_time_metrics (
    self)
```

7.23.1.6 test_optimize_positions()

```
test_real_time_analysis.TestRealTimeAnalyzer.test_optimize_positions (
    self)
```

7.23.1.7 test_plot_real_time_analysis()

```
test_real_time_analysis.TestRealTimeAnalyzer.test_plot_real_time_analysis (
    self)
```

7.23.1.8 test_update_positions()

```
test_real_time_analysis.TestRealTimeAnalyzer.test_update_positions (
    self)
```

7.23.2 Member Data Documentation**7.23.2.1 analyzer**

```
test_real_time_analysis.TestRealTimeAnalyzer.analyzer
```

Initial value:

```
= RealTimeAnalyzer(
    window_size=252,
    update_frequency=60,
    risk_threshold=0.02,
    rebalance_threshold=0.05,
    max_position_size=0.05
)
```

7.23.2.2 positions

`test_real_time_analysis.TestRealTimeAnalyzer.positions`

Initial value:

```
= {
    'AAPL': 0.3,
    'GOOGL': 0.2,
    'MSFT': 0.15,
    'AMZN': 0.15,
    'META': 0.2
}
```

7.23.2.3 price_data

`test_real_time_analysis.TestRealTimeAnalyzer.price_data`

Initial value:

```
= pd.DataFrame(
    np.random.normal(100, 10, (252, 5)),
    index=pd.date_range('2023-01-01', periods=252),
    columns=['AAPL', 'GOOGL', 'MSFT', 'AMZN', 'META']
)
```

7.23.2.4 returns

`test_real_time_analysis.TestRealTimeAnalyzer.returns`

Initial value:

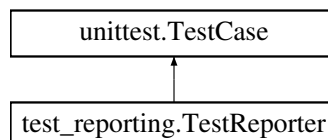
```
= pd.Series(
    np.random.normal(0, 0.01, 252),
    index=pd.date_range('2023-01-01', periods=252)
)
```

The documentation for this class was generated from the following file:

- [backend/tests/test_real_time_analysis.py](#)

7.24 test_reporting.TestReporter Class Reference

Inheritance diagram for `test_reporting.TestReporter`:



Public Member Functions

- [setUp](#) (self)
- [test_generate_performance_report](#) (self)
- [test_generate_portfolio_report](#) (self)
- [test_generate_risk_report](#) (self)
- [test_plot_performance](#) (self)
- [test_plot_portfolio_analysis](#) (self)
- [test_generate_html_report](#) (self)
- [test_save_report](#) (self)

Public Attributes

- [reporter](#)
- [returns](#)
- [positions](#)
- [predictions](#)
- [actuals](#)

7.24.1 Member Function Documentation

7.24.1.1 setUp()

```
test_reporting.TestReporter.setUp (
    self)
```

7.24.1.2 test_generate_html_report()

```
test_reporting.TestReporter.test_generate_html_report (
    self)
```

7.24.1.3 test_generate_performance_report()

```
test_reporting.TestReporter.test_generate_performance_report (
    self)
```

7.24.1.4 test_generate_portfolio_report()

```
test_reporting.TestReporter.test_generate_portfolio_report (
    self)
```

7.24.1.5 test_generate_risk_report()

```
test_reporting.TestReporter.test_generate_risk_report (
    self)
```

7.24.1.6 test_plot_performance()

```
test_reporting.TestReporter.test_plot_performance (
    self)
```

7.24.1.7 test_plot_portfolio_analysis()

```
test_reporting.TestReporter.test_plot_portfolio_analysis (
    self)
```

7.24.1.8 test_save_report()

```
test_reporting.TestReporter.test_save_report (
    self)
```

7.24.2 Member Data Documentation

7.24.2.1 actuals

```
test_reporting.TestReporter.actuals
Initial value:
= pd.Series(
    np.random.normal(0, 0.01, 252),
    index=pd.date_range('2023-01-01', periods=252)
)
```

7.24.2.2 positions

```
test_reporting.TestReporter.positions
Initial value:
= pd.DataFrame(
    np.random.normal(0, 0.1, (252, 5)),
    index=pd.date_range('2023-01-01', periods=252),
    columns=['AAPL', 'GOOGL', 'MSFT', 'AMZN', 'META']
)
```

7.24.2.3 predictions

test_reporting.TestReporter.predictions

Initial value:

```
= pd.Series(
    np.random.normal(0, 0.01, 252),
    index=pd.date_range('2023-01-01', periods=252)
)
```

7.24.2.4 reporter

test_reporting.TestReporter.reporter

Initial value:

```
= Reporter(
    window_size=252,
    report_frequency='D'
)
```

7.24.2.5 returns

test_reporting.TestReporter.returns

Initial value:

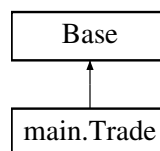
```
= pd.Series(
    np.random.normal(0, 0.01, 252),
    index=pd.date_range('2023-01-01', periods=252)
)
```

The documentation for this class was generated from the following file:

- [backend/tests/test_reporting.py](#)

7.25 main.Trade Class Reference

Inheritance diagram for main.Trade:



Static Public Attributes

- `id` = Column(Integer, primary_key=True, index=True)
- `symbol` = Column(String, index=True)
- `quantity` = Column(Float)
- `price` = Column(Float)
- `side` = Column(String)
- `timestamp` = Column(DateTime)
- `strategy` = Column(String)
- `status` = Column(String)

7.25.1 Member Data Documentation

7.25.1.1 id

```
main.Trade.id = Column(Integer, primary_key=True, index=True) [static]
```

7.25.1.2 price

```
main.Trade.price = Column(Float) [static]
```

7.25.1.3 quantity

```
main.Trade.quantity = Column(Float) [static]
```

7.25.1.4 side

```
main.Trade.side = Column(String) [static]
```

7.25.1.5 status

```
main.Trade.status = Column(String) [static]
```

7.25.1.6 strategy

```
main.Trade.strategy = Column(String) [static]
```

7.25.1.7 symbol

```
main.Trade.symbol = Column(String, index=True) [static]
```

7.25.1.8 timestamp

```
main.Trade.timestamp = Column(DateTime) [static]
```

The documentation for this class was generated from the following file:

- backend/[main.py](#)

7.26 strategy_framework.TradingStrategy Class Reference**Public Member Functions**

- `__init__` (self, str [name](#), Dict [parameters](#)=None)
- pd.Series [generate_signals](#) (self, pd.DataFrame data)
- Dict [calculate_metrics](#) (self, pd.Series returns)

Public Attributes

- [name](#) = name
- [parameters](#) = parameters or {}
- dict [metrics](#) = {}
- [correlation_matrix](#) = None

7.26.1 Constructor & Destructor Documentation**7.26.1.1 __init__()**

```
strategy_framework.TradingStrategy.__init__ (
    self,
    str name,
    Dict parameters = None)
```

7.26.2 Member Function Documentation**7.26.2.1 calculate_metrics()**

```
Dict strategy_framework.TradingStrategy.calculate_metrics (
    self,
    pd.Series returns)
```

Calculate strategy metrics

7.26.2.2 generate_signals()

```
pd.Series strategy_framework.TradingStrategy.generate_signals (
    self,
    pd.DataFrame data)
```

Generate trading signals

7.26.3 Member Data Documentation

7.26.3.1 correlation_matrix

```
strategy_framework.TradingStrategy.correlation_matrix = None
```

7.26.3.2 metrics

```
dict strategy_framework.TradingStrategy.metrics = {}
```

7.26.3.3 name

```
strategy_framework.TradingStrategy.name = name
```

7.26.3.4 parameters

```
strategy_framework.TradingStrategy.parameters = parameters or {}
```

The documentation for this class was generated from the following file:

- backend/strategies/[strategy_framework.py](#)

7.27 cost_analysis.TransactionCostAnalyzer Class Reference

Public Member Functions

- `__init__` (self, float [base_commission](#)=0.001, float [slippage_base](#)=0.0005, float [liquidity_threshold](#)=0.05, float [market_impact_exponent](#)=0.6, bool [cache_enabled](#)=True)
- float [calculate_commission_cost](#) (self, float trade_size, float price)
- float [calculate_slippage](#) (self, float trade_size, float price, float market_volume)
- float [calculate_market_impact](#) (self, float trade_size, float price, float market_volume)
- float [calculate_total_transaction_cost](#) (self, float trade_size, float price, float market_volume)
- pd.DataFrame [analyze_trade_costs](#) (self, pd.DataFrame trade_data)
- float [optimize_trade_size](#) (self, float target_size, float price, float market_volume, float max_cost_ratio=0.01)
- Dict[str, float] [get_cost_metrics](#) (self)
- None [plot_cost_analysis](#) (self, pd.DataFrame trade_data)

Public Attributes

- [base_commission](#) = base_commission
- [slippage_base](#) = slippage_base
- [liquidity_threshold](#) = liquidity_threshold
- [market_impact_exponent](#) = market_impact_exponent
- [cache_enabled](#) = cache_enabled
- dict [cost_cache](#) = {}

Protected Member Functions

- str [_get_cache_key](#) (self, float trade_size, float price, float market_volume)
- None [_cache_result](#) (self, str key, Any result)
- Optional[Any] [_get_cached_result](#) (self, str key)

7.27.1 Constructor & Destructor Documentation

7.27.1.1 `__init__()`

```
cost_analysis.TransactionCostAnalyzer.__init__ (
    self,
    float base_commission = 0.001,
    float slippage_base = 0.0005,
    float liquidity_threshold = 0.05,
    float market_impact_exponent = 0.6,
    bool cache_enabled = True)

Initialize transaction cost analyzer

Args:
    base_commission: Base commission rate
    slippage_base: Base slippage percentage
    liquidity_threshold: Threshold for liquidity impact
    market_impact_exponent: Exponent for market impact calculation
    cache_enabled: Enable caching of calculations
```

7.27.2 Member Function Documentation

7.27.2.1 `_cache_result()`

```
None cost_analysis.TransactionCostAnalyzer._cache_result (
    self,
    str key,
    Any result) [protected]

Cache result with TTL
```

7.27.2.2 `_get_cache_key()`

```
str cost_analysis.TransactionCostAnalyzer._get_cache_key (
    self,
    float trade_size,
    float price,
    float market_volume) [protected]

Generate cache key based on trade parameters
```

7.27.2.3 `_get_cached_result()`

```
Optional[Any] cost_analysis.TransactionCostAnalyzer._get_cached_result (
    self,
    str key) [protected]

Get cached result if valid
```

7.27.2.4 `analyze_trade_costs()`

```
pd.DataFrame cost_analysis.TransactionCostAnalyzer.analyze_trade_costs (
    self,
    pd.DataFrame trade_data)

Analyze transaction costs for a series of trades
```

7.27.2.5 calculate_commission_cost()

```
float cost_analysis.TransactionCostAnalyzer.calculate_commission_cost (
    self,
    float trade_size,
    float price)
```

Calculate commission cost with caching and optimization

7.27.2.6 calculate_market_impact()

```
float cost_analysis.TransactionCostAnalyzer.calculate_market_impact (
    self,
    float trade_size,
    float price,
    float market_volume)
```

Calculate market impact with caching and optimization

7.27.2.7 calculate_slippage()

```
float cost_analysis.TransactionCostAnalyzer.calculate_slippage (
    self,
    float trade_size,
    float price,
    float market_volume)
```

Calculate slippage with caching and optimization

7.27.2.8 calculate_total_transaction_cost()

```
float cost_analysis.TransactionCostAnalyzer.calculate_total_transaction_cost (
    self,
    float trade_size,
    float price,
    float market_volume)
```

Calculate total transaction cost with caching and optimization

7.27.2.9 get_cost_metrics()

```
Dict[str, float] cost_analysis.TransactionCostAnalyzer.get_cost_metrics (
    self)
```

Get current cost parameters

7.27.2.10 optimize_trade_size()

```
float cost_analysis.TransactionCostAnalyzer.optimize_trade_size (
    self,
    float target_size,
    float price,
    float market_volume,
    float max_cost_ratio = 0.01)
```

Optimize trade size to minimize transaction costs

7.27.2.11 plot_cost_analysis()

```
None cost_analysis.TransactionCostAnalyzer.plot_cost_analysis (
    self,
    pd.DataFrame trade_data)
Plot cost analysis results
```

7.27.3 Member Data Documentation

7.27.3.1 base_commission

```
cost_analysis.TransactionCostAnalyzer.base_commission = base_commission
```

7.27.3.2 cache_enabled

```
cost_analysis.TransactionCostAnalyzer.cache_enabled = cache_enabled
```

7.27.3.3 cost_cache

```
dict cost_analysis.TransactionCostAnalyzer.cost_cache = {}
```

7.27.3.4 liquidity_threshold

```
cost_analysis.TransactionCostAnalyzer.liquidity_threshold = liquidity_threshold
```

7.27.3.5 market_impact_exponent

```
cost_analysis.TransactionCostAnalyzer.market_impact_exponent = market_impact_exponent
```

7.27.3.6 slippage_base

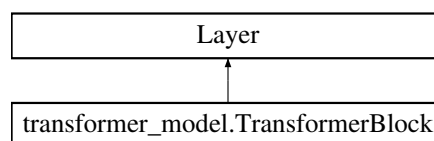
```
cost_analysis.TransactionCostAnalyzer.slippage_base = slippage_base
```

The documentation for this class was generated from the following file:

- backend/[cost_analysis.py](#)

7.28 transformer_model.TransformerBlock Class Reference

Inheritance diagram for transformer_model.TransformerBlock:



Public Member Functions

- [__init__](#) (self, embed_dim, num_heads, ff_dim, rate=0.1)
- [call](#) (self, inputs, training)

Public Attributes

- [att](#) = [MultiHeadSelfAttention](#)(embed_dim, num_heads)
- [ffn](#)
- [layernorm1](#) = [LayerNormalization](#)(epsilon=1e-6)
- [layernorm2](#) = [LayerNormalization](#)(epsilon=1e-6)
- [dropout1](#) = [Dropout](#)(rate)
- [dropout2](#) = [Dropout](#)(rate)

7.28.1 Constructor & Destructor Documentation

7.28.1.1 `__init__()`

```
transformer_model.TransformerBlock.__init__ (
    self,
    embed_dim,
    num_heads,
    ff_dim,
    rate = 0.1)
```

7.28.2 Member Function Documentation

7.28.2.1 `call()`

```
transformer_model.TransformerBlock.call (
    self,
    inputs,
    training)
```

7.28.3 Member Data Documentation

7.28.3.1 `att`

```
transformer_model.TransformerBlock.att = MultiHeadSelfAttention(embed_dim, num_heads)
```

7.28.3.2 `dropout1`

```
transformer_model.TransformerBlock.dropout1 = Dropout(rate)
```

7.28.3.3 `dropout2`

```
transformer_model.TransformerBlock.dropout2 = Dropout(rate)
```

7.28.3.4 `ffn`

```
transformer_model.TransformerBlock.ffn
```

Initial value:

```
= tf.keras.Sequential([
    Dense(ff_dim, activation="relu"),
    Dense(embed_dim),
])
```

7.28.3.5 `layernorm1`

```
transformer_model.TransformerBlock.layernorm1 = LayerNormalization(epsilon=1e-6)
```

7.28.3.6 `layernorm2`

```
transformer_model.TransformerBlock.layernorm2 = LayerNormalization(epsilon=1e-6)
```

The documentation for this class was generated from the following file:

- [backend/ai_models/transformer_model.py](#)

7.29 `transformer_model.TransformerModel` Class Reference

Public Member Functions

- `__init__` (self, Tuple[int, int] [input_shape](#)=(60, 10), int [embed_dim](#)=64, int [num_heads](#)=4, int [ff_dim](#)=32, int [num_transformer_blocks](#)=3, float [dropout_rate](#)=0.2, float [learning_rate](#)=0.001, int [batch_size](#)=32, int [epochs](#)=100, int [patience](#)=10, int [verbose](#)=1)
- None [build_model](#) (self)

- Tuple[np.ndarray, np.ndarray] [prepare_data](#) (self, pd.DataFrame data)
- None [train](#) (self, np.ndarray X_train, np.ndarray y_train, Optional[np.ndarray] X_val=None, Optional[np.ndarray] y_val=None)
- np.ndarray [predict](#) (self, np.ndarray X)
- Dict[str, float] [evaluate](#) (self, np.ndarray X_test, np.ndarray y_test)
- None [save_model](#) (self, str path)
- None [load_model](#) (self, str path)
- Dict[str, List[float]] [get_metrics](#) (self)
- str [get_model_summary](#) (self)

Public Attributes

- [input_shape](#) = input_shape
- [embed_dim](#) = embed_dim
- [num_heads](#) = num_heads
- [ff_dim](#) = ff_dim
- [num_transformer_blocks](#) = num_transformer_blocks
- [dropout_rate](#) = dropout_rate
- [learning_rate](#) = learning_rate
- [batch_size](#) = batch_size
- [epochs](#) = epochs
- [patience](#) = patience
- [verbose](#) = verbose
- None [model](#) = None
- [scaler](#) = StandardScaler()
- List[tf.keras.callbacks.Callback] [callbacks](#) = self._create_callbacks()
- Optional[datetime] [last_update](#) = None
- dict [metrics](#)

Protected Member Functions

- List[tf.keras.callbacks.Callback] [_create_callbacks](#) (self)

7.29.1 Constructor & Destructor Documentation

7.29.1.1 `__init__()`

```
transformer_model.TransformerModel.__init__ (
    self,
    Tuple[int, int] input_shape = (60, 10),
    int embed_dim = 64,
    int num_heads = 4,
    int ff_dim = 32,
    int num_transformer_blocks = 3,
    float dropout_rate = 0.2,
    float learning_rate = 0.001,
    int batch_size = 32,
    int epochs = 100,
    int patience = 10,
    int verbose = 1)
```

Initialize Transformer model for trading predictions

Args:

```
input_shape: Shape of input data (timesteps, features)
embed_dim: Dimension of the embedding space
num_heads: Number of attention heads
ff_dim: Dimension of the feed-forward network
num_transformer_blocks: Number of transformer blocks
```

dropout_rate: Dropout rate for regularization
learning_rate: Learning rate for optimizer
batch_size: Batch size for training
epochs: Maximum number of epochs
patience: Patience for early stopping
verbose: Verbosity level

7.29.2 Member Function Documentation

7.29.2.1 `_create_callbacks()`

```
List[tf.keras.callbacks.Callback] transformer_model.TransformerModel._create_callbacks (  
    self) [protected]
```

Create callbacks for training

7.29.2.2 `build_model()`

```
None transformer_model.TransformerModel.build_model (  
    self)
```

Build the Transformer model architecture

7.29.2.3 `evaluate()`

```
Dict[str, float] transformer_model.TransformerModel.evaluate (  
    self,  
    np.ndarray X_test,  
    np.ndarray y_test)
```

Evaluate model performance

7.29.2.4 `get_metrics()`

```
Dict[str, List[float]] transformer_model.TransformerModel.get_metrics (  
    self)
```

Get training metrics history

7.29.2.5 `get_model_summary()`

```
str transformer_model.TransformerModel.get_model_summary (  
    self)
```

Get model architecture summary

7.29.2.6 `load_model()`

```
None transformer_model.TransformerModel.load_model (  
    self,  
    str path)
```

Load a pre-trained model

7.29.2.7 `predict()`

```
np.ndarray transformer_model.TransformerModel.predict (  
    self,  
    np.ndarray X)
```

Make predictions using the trained model

7.29.2.8 prepare_data()

```
Tuple[np.ndarray, np.ndarray] transformer_model.TransformerModel.prepare_data (  
    self,  
    pd.DataFrame data)
```

Prepare data for Transformer input

7.29.2.9 save_model()

```
None transformer_model.TransformerModel.save_model (  
    self,  
    str path)
```

Save the trained model

7.29.2.10 train()

```
None transformer_model.TransformerModel.train (  
    self,  
    np.ndarray X_train,  
    np.ndarray y_train,  
    Optional[np.ndarray] X_val = None,  
    Optional[np.ndarray] y_val = None)
```

Train the Transformer model

7.29.3 Member Data Documentation

7.29.3.1 batch_size

```
transformer_model.TransformerModel.batch_size = batch_size
```

7.29.3.2 callbacks

```
List[tf.keras.callbacks.Callback] transformer_model.TransformerModel.callbacks = self._↵  
create_callbacks()
```

7.29.3.3 dropout_rate

```
transformer_model.TransformerModel.dropout_rate = dropout_rate
```

7.29.3.4 embed_dim

```
transformer_model.TransformerModel.embed_dim = embed_dim
```

7.29.3.5 epochs

```
transformer_model.TransformerModel.epochs = epochs
```

7.29.3.6 ff_dim

```
transformer_model.TransformerModel.ff_dim = ff_dim
```

7.29.3.7 input_shape

```
transformer_model.TransformerModel.input_shape = input_shape
```

7.29.3.8 last_update

```
Optional[datetime] transformer_model.TransformerModel.last_update = None
```

7.29.3.9 learning_rate

```
transformer_model.TransformerModel.learning_rate = learning_rate
```

7.29.3.10 metrics

```
dict transformer_model.TransformerModel.metrics
```

Initial value:

```
= {
    'accuracy': [],
    'loss': [],
    'val_accuracy': [],
    'val_loss': []
}
```

7.29.3.11 model

```
None transformer_model.TransformerModel.model = None
```

7.29.3.12 num_heads

```
transformer_model.TransformerModel.num_heads = num_heads
```

7.29.3.13 num_transformer_blocks

```
transformer_model.TransformerModel.num_transformer_blocks = num_transformer_blocks
```

7.29.3.14 patience

```
transformer_model.TransformerModel.patience = patience
```

7.29.3.15 scaler

```
transformer_model.TransformerModel.scaler = StandardScaler()
```

7.29.3.16 verbose

```
transformer_model.TransformerModel.verbose = verbose
```

The documentation for this class was generated from the following file:

- backend/ai_models/[transformer_model.py](#)

Chapter 8

File Documentation

8.1 backend/ai_models/allocator.py File Reference

Namespaces

- namespace [allocator](#)

8.2 backend/ai_models/feature_engineering.py File Reference

Classes

- class [feature_engineering.FeatureEngineer](#)

Namespaces

- namespace [feature_engineering](#)

Variables

- [feature_engineering.logger](#) = logging.getLogger(__name__)
- [feature_engineering.feature_engineer](#) = [FeatureEngineer](#)()

8.3 backend/ai_models/lstm_model.py File Reference

Classes

- class [lstm_model.LSTMModel](#)

Namespaces

- namespace [lstm_model](#)

Variables

- [lstm_model.logger](#) = logging.getLogger(__name__)
- [lstm_model.lstm](#)

8.4 backend/ai_models/market_predictor.py File Reference

Classes

- class [market_predictor.MarketPredictor](#)

Namespaces

- namespace [market_predictor](#)

Variables

- [market_predictor.logger](#) = logging.getLogger(__name__)
- [market_predictor.market_predictor](#) = [MarketPredictor](#)()

8.5 backend/ai_models/meta_learner.py File Reference

Classes

- class [meta_learner.MetaLearner](#)

Namespaces

- namespace [meta_learner](#)

Variables

- [meta_learner.logger](#) = logging.getLogger(__name__)
- [meta_learner.meta](#)

8.6 backend/ai_models/meta_reinforcement.py File Reference

Classes

- class [meta_reinforcement.MetaReinforcementLearner](#)

Namespaces

- namespace [meta_reinforcement](#)

8.7 backend/ai_models/risk_manager.py File Reference

Classes

- class [risk_manager.RiskManager](#)

Namespaces

- namespace [risk_manager](#)

Variables

- [risk_manager.logger](#) = logging.getLogger(__name__)
- [risk_manager.risk_manager](#) = [RiskManager](#)()

8.8 backend/ai_models/train_lstm.py File Reference

Namespaces

- namespace [train_lstm](#)

Functions

- [train_lstm.train_lstm_model](#) (prices)

8.9 backend/ai_models/transformer_model.py File Reference

Classes

- class [transformer_model.MultiHeadSelfAttention](#)
- class [transformer_model.TransformerBlock](#)
- class [transformer_model.TransformerModel](#)

Namespaces

- namespace [transformer_model](#)

Variables

- [transformer_model.logger](#) = logging.getLogger(__name__)
- [transformer_model.transformer](#)

8.10 backend/api/manual_trade.py File Reference

Namespaces

- namespace [manual_trade](#)

Functions

- [manual_trade.manual_trade](#) (Request request)

Variables

- [manual_trade.router](#) = APIRouter()

8.11 backend/backtesting.py File Reference

Classes

- class [backtesting.Backtester](#)

Namespaces

- namespace [backtesting](#)

Variables

- [backtesting.logger](#) = logging.getLogger(__name__)
- [backtesting.backtester](#) = [Backtester](#)()

8.12 backend/config/celery_config.py File Reference

Namespaces

- namespace [celery_config](#)

8.13 backend/config/settings.py File Reference

Classes

- class [settings.Settings](#)

Namespaces

- namespace [settings](#)

Variables

- [settings.settings](#) = [Settings\(\)](#)

8.14 backend/cost_analysis.py File Reference

Classes

- class [cost_analysis.TransactionCostAnalyzer](#)

Namespaces

- namespace [cost_analysis](#)

Functions

- float [cost_analysis._fast_commission_cost](#) (float trade_size, float price, float base_commission)
- float [cost_analysis._fast_slippage](#) (float trade_size, float price, float market_volume, float slippage_base, float liquidity_threshold)
- float [cost_analysis._fast_market_impact](#) (float trade_size, float price, float market_volume, float market_impact_exponent)
- float [cost_analysis._fast_total_transaction_cost](#) (float trade_size, float price, float market_volume, float base_commission, float slippage_base, float liquidity_threshold, float market_impact_exponent)

Variables

- [cost_analysis.logger](#) = logging.getLogger(__name__)
- dict [cost_analysis.cost_cache](#) = {}
- [cost_analysis.last_cache_update](#) = datetime.min
- int [cost_analysis.CACHE_TTL](#) = 300
- [cost_analysis.cost_analyzer](#) = [TransactionCostAnalyzer\(\)](#)

8.15 backend/data/websocket_stream.py File Reference

Namespaces

- namespace [websocket_stream](#)

8.16 backend/integrations/nlp_sentiment.py File Reference

Namespaces

- namespace [nlp_sentiment](#)

Functions

- [nlp_sentiment.analyze_foreign_news](#) (texts, target_lang='en')

Variables

- [nlp_sentiment.translator](#) = Translator()
- [nlp_sentiment.sentiment_pipeline](#) = pipeline("sentiment-analysis")

8.17 backend/integrations/telegram_alerts.py File Reference

Namespaces

- namespace [telegram_alerts](#)

Functions

- [telegram_alerts.send_telegram_message](#) (str message)

8.18 backend/integrations/twelve_data.py File Reference

Namespaces

- namespace [twelve_data](#)

Functions

- [twelve_data.fetch_realtime_data](#) (symbols)

8.19 backend/main.py File Reference

Classes

- class [main.Trade](#)
- class [main.PerformanceMetric](#)
- class [main.RiskMetric](#)
- class [main.RequestValidator](#)
- class [main.CacheManager](#)
- class [main.RateLimiter](#)
- class [main.PerformanceMonitor](#)

Namespaces

- namespace [main](#)

Functions

- [main.log_ai_decision](#) (str decision_type, dict details)
- bool [main.check_ai_safety](#) (dict trade_details)
- bool [main.require_user_confirmation](#) (dict decision)
- [main.ping_connection](#) (dbapi_connection, connection_record, connection_proxy)
- [main.get_db](#) ()
- [main.get_cache](#) (namespace='default')
- [main.get_redis_pool](#) ()
- [main.get_memcached_client](#) ()
- [main.custom_exception_handler](#) (Request request, Exception exc)
- [main.add_security_headers](#) (Request request, call_next)
- [main.monitor_performance](#) (Request request, call_next)
- [main.update_metrics](#) (str endpoint, float response_time)
- [main.circuit_breaker](#) (fn, *args, **kwargs)
- [main.run_background_task](#) (task)
- [main.validate_request](#) (Request request, call_next)
- [main.add_process_time_header](#) (Request request, call_next)
- [main.health_check](#) ()
- [main.get_metrics](#) ()
- [main.read_root](#) ()
- [main.run_all_strategies](#) ()

Variables

- dict `main.AI_CONFIG`
- dict `main.ai_safety_monitor`
- `main.level`
- `main.format`
- `main.handlers`
- `main.datefmt`
- `main.logger` = `logging.getLogger("quantum_trader")`
- `main.jaeger_exporter`
- `main.span_processor` = `BatchSpanProcessor(jaeger_exporter)`
- str `main.DATABASE_URL` = "postgresql+asyncpg://user:password@localhost/quantum_trader"
- `main.engine`
- `main.SessionLocal`
- `main.Base` = `declarative_base()`
- dict `main.CACHE_CONFIG`
- `main.app`
- int `main.RATE_LIMIT_WINDOW` = 60
- int `main.RATE_LIMIT_REQUESTS` = 100
- `main.rate_limit_cache` = `TTLCache(maxsize=1000, ttl=RATE_LIMIT_WINDOW)`
- `main.SECRET_KEY` = `secrets.token_hex(32)`
- str `main.ALGORITHM` = "HS256"
- int `main.ACCESS_TOKEN_EXPIRE_MINUTES` = 30
- `main.pwd_context` = `CryptContext(schemes=["bcrypt"], deprecated="auto")`
- `main.redis_pool` = None
- `main.memcached_client` = None
- `main.dsn`
- `main.traces_sample_rate`
- `main.environment`
- `main.integrations`
- `main.before_send`
- int `main.CIRCUIT_TIMEOUT` = 30000
- int `main.last_failure_time` = 0
- bool `main.circuit_open` = False
- `main.background_tasks` = `set()`
- `main.cache_manager` = `CacheManager()`
- `main.allow_origins`
- `main.allow_credentials`
- `main.allow_methods`
- `main.allow_headers`
- `main.expose_headers`
- `main.allow_origin_regex`
- `main.max_age`
- `main.minimum_size`
- `main.compresslevel`
- `main.server_side`
- `main.client_side`
- `main.allowed_hosts`
- `main.except_hosts`
- `main.status_code`
- `main.ssl_host`
- `main.ssl_port`
- `main.secret_key`
- `main.session_cookie`
- `main.same_site`

- [main.https_only](#)
- [main.cookie_domain](#)
- [main.headers](#)
- [main.max_requests](#)
- [main.window](#)
- [main.key_func](#)
- [main.storage](#)
- [main.on_limit_exceeded](#)
- [main.threshold](#)
- [main.metrics_store](#)
- [main.on_slow_request](#)
- [main.backend](#)
- [main.on_error](#)
- [main.cache](#)
- [main.cache_key_func](#)
- [main.cache_ttl](#)
- [main.cache_bypass_func](#)
- [main.tracer](#)
- [main.span_name_func](#)
- [main.trace_id_header](#)
- [main.span_id_header](#)
- [main.log_format](#)
- [main.log_level](#)
- [main.include_headers](#)
- [main.include_body](#)
- [main.validators](#)
- [main.failure_threshold](#)
- [main.reset_timeout](#)
- [main.on_circuit_open](#)
- dict [main.market_data_cache](#) = {}
- [main.last_cache_update](#) = datetime.min
- int [main.CACHE_TTL](#) = 300

8.20 backend/market_simulation.py File Reference

Classes

- class [market_simulation.MarketSimulator](#)

Namespaces

- namespace [market_simulation](#)

Variables

- [market_simulation.logger](#) = logging.getLogger(__name__)
- [market_simulation.market_simulator](#) = [MarketSimulator](#)()

8.21 backend/metrics.py File Reference

Classes

- class [metrics.Metrics](#)

Namespaces

- namespace [metrics](#)

Functions

- float [metrics._fast_sharpe_ratio](#) (np.ndarray returns, float risk_free_rate)
- float [metrics._fast_max_drawdown](#) (np.ndarray cumulative)
- Dict [metrics._fast_basic_metrics](#) (np.ndarray returns, float risk_free_rate)

Variables

- [metrics.logger](#) = logging.getLogger(__name__)
- dict [metrics.metrics_cache](#) = {}
- [metrics.last_cache_update](#) = datetime.min
- int [metrics.CACHE_TTL](#) = 300

8.22 backend/ai_models/portfolio_optimizer.py File Reference

Classes

- class [portfolio_optimizer.PortfolioOptimizer](#)

Namespaces

- namespace [portfolio_optimizer](#)

Variables

- [portfolio_optimizer.logger](#) = logging.getLogger(__name__)
- [portfolio_optimizer.portfolio_optimizer](#) = [PortfolioOptimizer](#)()

8.23 backend/portfolio_optimizer.py File Reference

Classes

- class [portfolio_optimizer.PortfolioOptimizer](#)

Namespaces

- namespace [portfolio_optimizer](#)

8.24 backend/real_time_analysis.py File Reference

Classes

- class [real_time_analysis.RealTimeAnalyzer](#)

Namespaces

- namespace [real_time_analysis](#)

Variables

- [real_time_analysis.logger](#) = logging.getLogger(__name__)
- [real_time_analysis.real_time_analyzer](#) = [RealTimeAnalyzer](#)()

8.25 backend/reporting.py File Reference

Namespaces

- namespace [reporting](#)

Variables

- [reporting.logger](#) = logging.getLogger(__name__)

8.26 backend/risk_management.py File Reference

Classes

- class [risk_management.RiskManager](#)

Namespaces

- namespace [risk_management](#)

Variables

- [risk_management.logger](#) = logging.getLogger(__name__)
- [risk_management.risk_manager](#) = [RiskManager](#)()

8.27 backend/strategies/mean_reversion.py File Reference

Namespaces

- namespace [mean_reversion](#)

Functions

- [mean_reversion.mean_reversion_strategy](#) (data)

8.28 backend/strategies/momentum.py File Reference

Namespaces

- namespace [momentum](#)

Functions

- [momentum.momentum_strategy](#) (data)

8.29 backend/strategies/strategy_framework.py File Reference

Classes

- class [strategy_framework.TradingStrategy](#)
- class [strategy_framework.StrategyFramework](#)

Namespaces

- namespace [strategy_framework](#)

Variables

- [strategy_framework.logger](#) = logging.getLogger(__name__)

8.30 backend/tasks/backtester.py File Reference

Namespaces

- namespace [backtester](#)

Functions

- [backtester.run_backtest](#) (data, strategy_fn, strategy_name, output_path='data/backtest_results.json')

8.31 backend/tasks/celery_worker.py File Reference

Namespaces

- namespace [celery_worker](#)

8.32 backend/tasks/strategy_runner.py File Reference

Namespaces

- namespace [strategy_runner](#)

8.33 backend/tests/test_real_time_analysis.py File Reference

Classes

- class [test_real_time_analysis.TestRealTimeAnalyzer](#)

Namespaces

- namespace [test_real_time_analysis](#)

8.34 backend/tests/test_reporting.py File Reference

Classes

- class [test_reporting.TestReporter](#)

Namespaces

- namespace [test_reporting](#)

8.35 backend/utils/data_loader.py File Reference

Namespaces

- namespace [data_loader](#)

Functions

- [data_loader.load_realtime_data](#) ()

8.36 backend/utils/latency_optimizer.py File Reference

Namespaces

- namespace [latency_optimizer](#)

Functions

- [latency_optimizer.measure_latency](#) ()

Variables

- dict [latency_optimizer.API_ENDPOINTS](#)

8.37 backend/utils/legal.py File Reference

Classes

- class [legal.LegalProtector](#)

Namespaces

- namespace [legal](#)

8.38 backend/utils/redis_client.py File Reference

Namespaces

- namespace [redis_client](#)

8.39 backend/utils/send_push.py File Reference

Namespaces

- namespace [send_push](#)

Functions

- [send_push.send_mobile_push](#) (title, body)

8.40 backend/utils/trade_executor.py File Reference

Namespaces

- namespace [trade_executor](#)

Functions

- [trade_executor.execute_trade](#) (signals)

8.41 docs/disclaimer.txt File Reference

Functions

- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential [damages whatsoever](#) (including, without limitation, [damages for](#) loss of business profits, business interruption, loss of business information, or any other pecuniary loss) arising out of the use of or inability to use this software

Variables

- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any [kind](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or [implied](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of [merchantability](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular [purpose](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any [special](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any [incidental](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible [for](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible [developers](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible and contributors from any [claims](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY Users are solely responsible and contributors from any [damages](#)
- by [copyright](#) law and international treaties Unauthorized reproduction or [distribution](#) is prohibited GOVERNING LAW This disclaimer shall be governed by and construed in accordance with the laws of the jurisdiction in which the software is used ENTIRE AGREEMENT This disclaimer constitutes the entire agreement between the parties regarding the use of QuantumTraderAI and supersedes all prior agreements and understandings MODIFICATIONS The authors reserve the right to modify this disclaimer at any time without notice SEVERABILITY If any provision of this disclaimer is found to be invalid or [unenforceable](#)

8.41.1 Function Documentation

8.41.1.1 whatsoever()

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential [damages](#) whatsoever (

```

    including ,
    without limitation,
```

damages for loss of business profits,
business interruption,
loss of business information,
or any other pecuniary loss)

8.41.2 Variable Documentation

8.41.2.1 claims

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY Users are solely responsible and contributors from any claims

8.41.2.2 damages

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY Users are solely responsible and contributors from any damages

8.41.2.3 developers

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY Users are solely responsible developers

8.41.2.4 for

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY Users are solely responsible for

8.41.2.5 implied

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or implied

8.41.2.6 incidental

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any incidental

8.41.2.7 kind

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any kind

8.41.2.8 merchantability

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of merchantability

8.41.2.9 purpose

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular purpose

8.41.2.10 special

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any special

8.41.2.11 unenforceable

by [copyright](#) law and international treaties Unauthorized reproduction or [distribution](#) is prohibited GOVERNING LAW This disclaimer shall be governed by and construed in accordance with the laws of the jurisdiction in which the software is used ENTIRE AGREEMENT This disclaimer constitutes the entire agreement between the parties regarding the use of QuantumTraderAI and supersedes all prior agreements and understandings MODIFICATIONS The authors reserve the right to modify this disclaimer at any time without notice SEVERABILITY If any provision of this disclaimer is found to be invalid or unenforceable [protected]

8.42 docs/README.md File Reference

8.43 README.md File Reference

8.44 docs/user_agreement.txt File Reference

Variables

- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable for any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible [for](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable for any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible [for](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable for any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible [for](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable for any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible [for](#)

- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable for any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible [for](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable for any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible [for](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY Users are solely responsible and contributors from any [damages](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any [incidental](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any [special](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY Users are solely responsible and contributors from any [damages](#)
- DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible and contributors from any [claims](#)
- by [copyright](#) law and international treaties Unauthorized reproduction or [distribution](#) is prohibited GOVERNING LAW This disclaimer shall be governed by and construed in accordance with the laws of the jurisdiction in which the software is used ENTIRE AGREEMENT This disclaimer constitutes the entire agreement between the parties regarding the use of QuantumTraderAI and supersedes all prior agreements and understandings MODIFICATIONS The authors reserve the right to modify this disclaimer at any time without notice SEVERABILITY If any provision of this disclaimer is found to be invalid or [unenforceable](#)

8.44.1 Variable Documentation

8.44.1.1 claims

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible and contributors from any [claims](#)

8.44.1.2 damages [1/2]

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any indirect or consequential

damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY
Users are solely responsible and contributors from any damages

8.44.1.3 damages [2/2]

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY
Users are solely responsible and contributors from any damages

8.44.1.4 for [1/6]

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY
Users are solely responsible for

8.44.1.5 for [2/6]

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY
Users are solely responsible for

8.44.1.6 for [3/6]

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY
Users are solely responsible for

8.44.1.7 for [4/6]

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY
Users are solely responsible for

8.44.1.8 for [5/6]

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the implied warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or copyright holders be liable for any indirect or consequential damages even if the authors have been advised of the possibility of such damages USER RESPONSIBILITY
Users are solely responsible for

8.44.1.9 for [6/6]

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness for a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable for any indirect or consequential [damages](#) even if the authors have been advised of the possibility of such [damages](#) USER RESPONSIBILITY Users are solely responsible for

8.44.1.10 incidental

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any incidental

8.44.1.11 special

DISCLAIMER OF LIABILITY NO WARRANTY The QuantumTraderAI software and related materials are provided as is without warranty of any either express or including but not limited to the [implied](#) warranties of fitness [for](#) a particular or non infringement LIMITATION OF LIABILITY In no event shall the authors or [copyright](#) holders be liable [for](#) any special

8.44.1.12 unenforceable

by [copyright](#) law and international treaties Unauthorized reproduction or [distribution](#) is prohibited GOVERNING LAW This disclaimer shall be governed by and construed in accordance with the laws of the jurisdiction in which the software is used ENTIRE AGREEMENT This disclaimer constitutes the entire agreement between the parties regarding the use of QuantumTraderAI and supersedes all prior agreements and understandings MODIFICATIONS The authors reserve the right to modify this disclaimer at any time without notice SEVERABILITY If any provision of this disclaimer is found to be invalid or unenforceable

8.45 LICENSE.txt File Reference

Functions

- [Copyright](#) (c) 2025 Breng Smith All rights reserved. This software and its components are the confidential and proprietary property of Breng Smith. Unauthorized copying

Variables

- [modification](#)
- [distribution](#)
- [publication](#)
- or use of this software or any part of [it](#)
- or use of this software or any part of via any [medium](#)
- or use of this software or any part of via any is strictly prohibited No license or right is granted to any person or entity under any [patent](#)
- or use of this software or any part of via any is strictly prohibited No license or right is granted to any person or entity under any [copyright](#)
- or use of this software or any part of via any is strictly prohibited No license or right is granted to any person or entity under any trade [secret](#)

8.45.1 Function Documentation

8.45.1.1 Copyright()

```
Copyright (
    c )
```

8.45.2 Variable Documentation

8.45.2.1 copyright

```
or use of this software or any part of via any is strictly prohibited No license or right is
granted to any person or entity under any copyright
```

8.45.2.2 distribution

```
distribution
```

8.45.2.3 it

```
or use of this software or any part of it
```

8.45.2.4 medium

```
or use of this software or any part of via any medium
```

8.45.2.5 modification

```
modification
```

8.45.2.6 patent

```
or use of this software or any part of via any is strictly prohibited No license or right is
granted to any person or entity under any patent
```

8.45.2.7 publication

```
publication
```

8.45.2.8 secret

```
or use of this software or any part of via any is strictly prohibited No license or right is
granted to any person or entity under any trade secret
```

8.46 backend/requirements.txt File Reference

8.47 requirements.txt File Reference